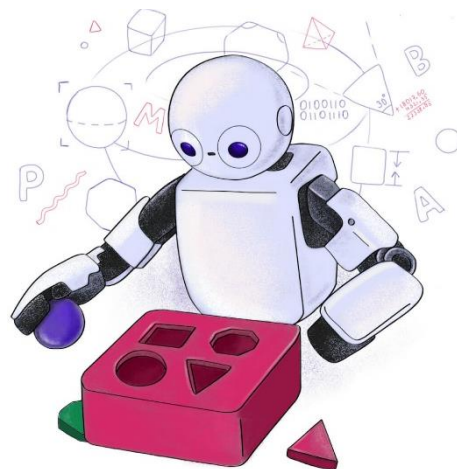


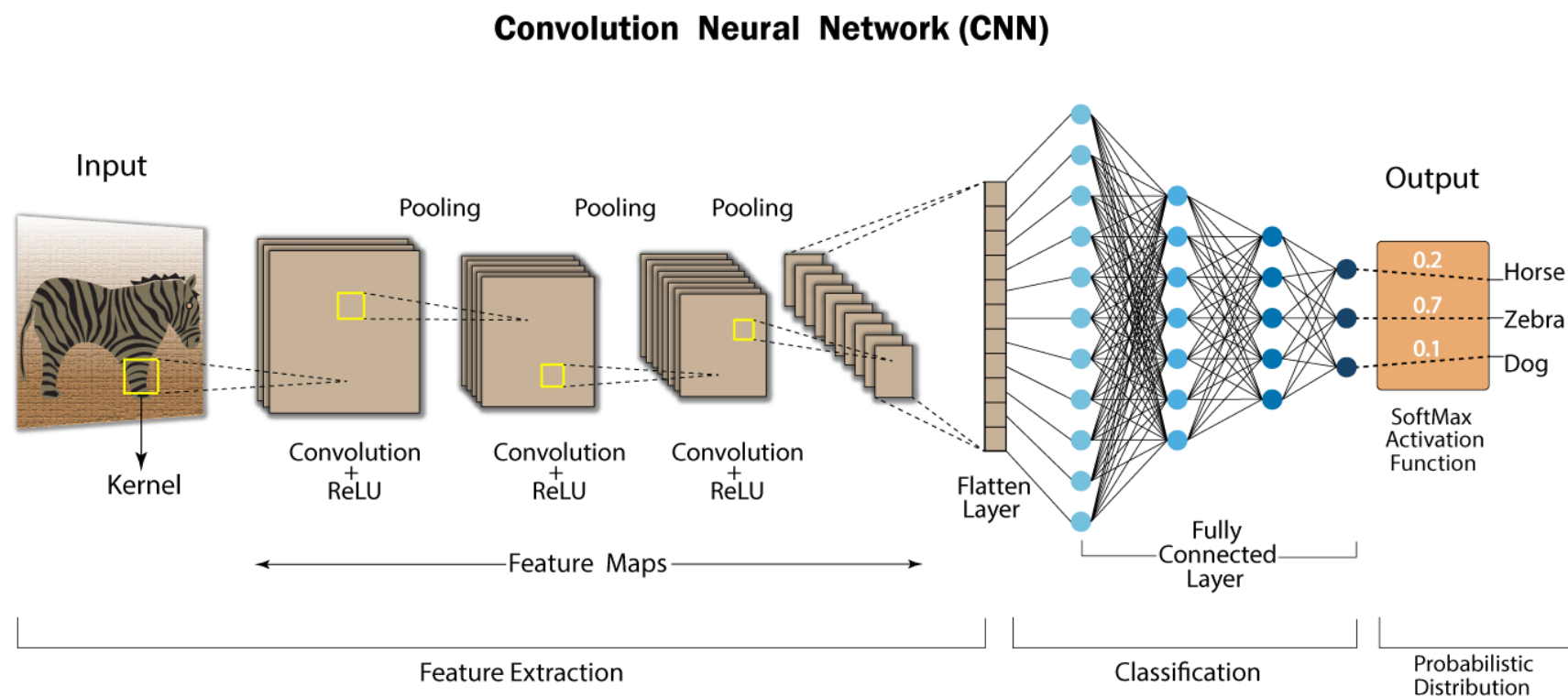
# C24 – Inteligência Artificial: *Redes Neurais Convolucionais*





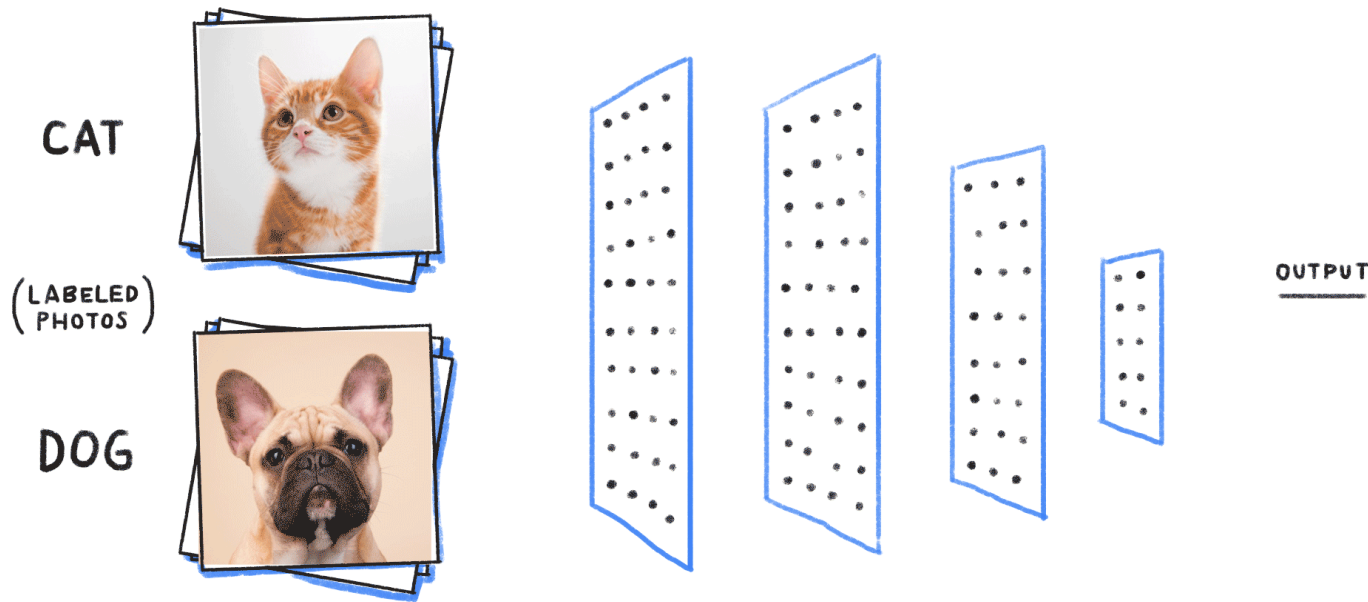
- Até agora, nossas redes neurais continham apenas **camadas densas**.
- Porém, um outro tipo muito importante são as **camadas convolucionais**.
- Essas camadas formam as *Convolutional Neural Networks* (CNNs).

# DNNs versus CNNs



- A **principal diferença** para uma DNN é que ao invés de **aprender os pesos** das camadas densas, uma **CNN aprende** os valores de **filtros de convolução** (ou **kernels**) que “compreendem” o conteúdo de imagens.

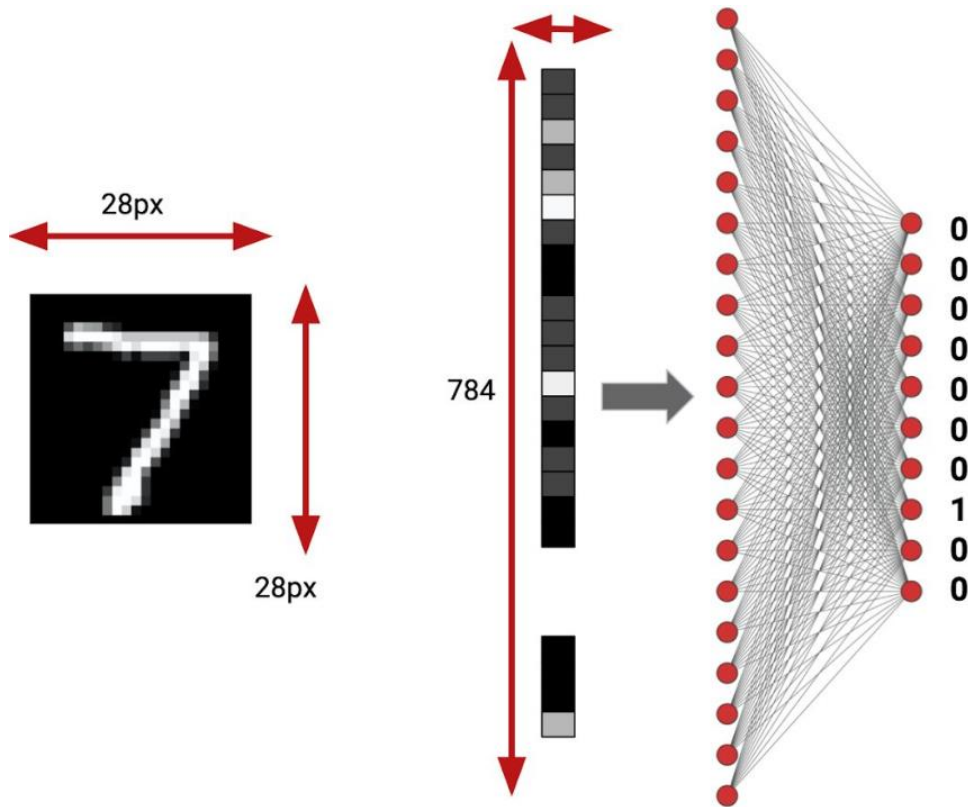
# O que vamos ver?



- CNNs são usadas em tarefas de **visão computacional**, como, por exemplo
  - classificação de imagens,
  - detecção de objetos,
  - segmentação de objetos,
  - reconhecimento facial e
  - análise médica.

Por que não usamos DNNs?

# Dados tabulares

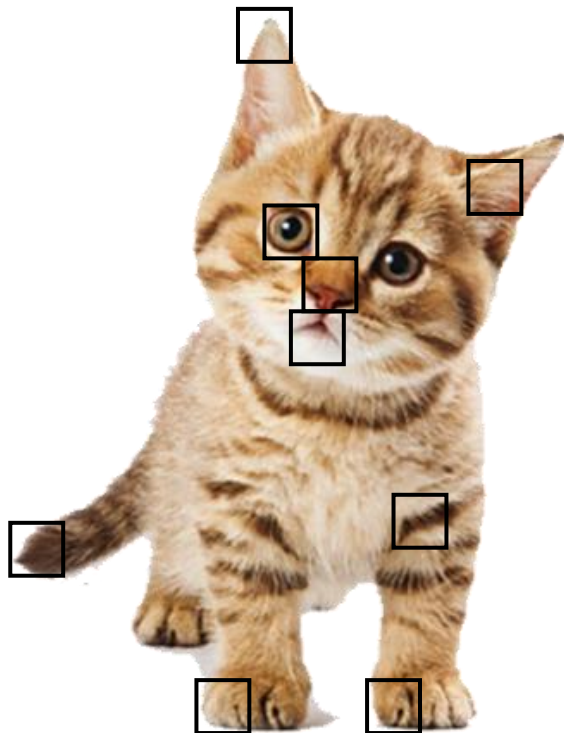


- DNNs são **ideais para dados tabulares**, onde os **exemplos são representados por linhas e os atributos por colunas**.
- O objetivo de uma DNN é **aprender relações entre atributos**, mas sem **presumir uma estrutura espacial** entre eles.
- Elas **não consideram a estrutura espacial dos dados**.
  - Ou seja, a posição espacial não influencia no resultado.

# Localidade de referência



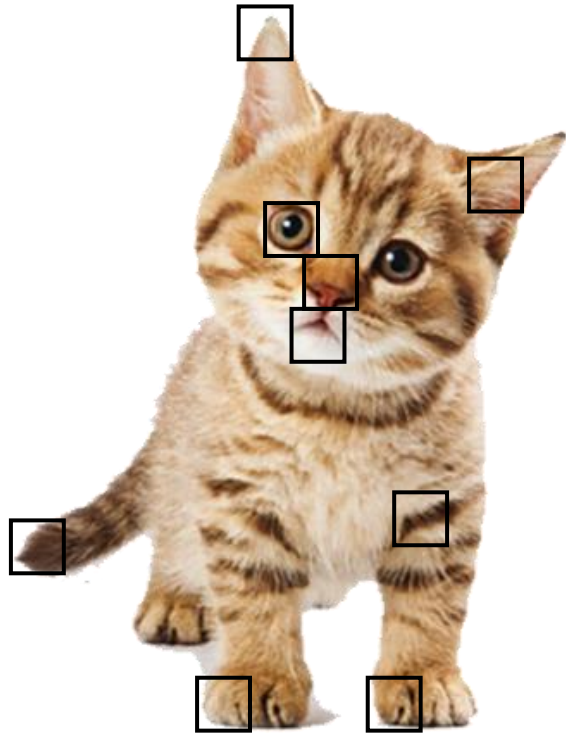
DNN: neurônios recebem todos os *pixels* como entrada



CNN: neurônios recebem apenas uma parte dos *pixels*

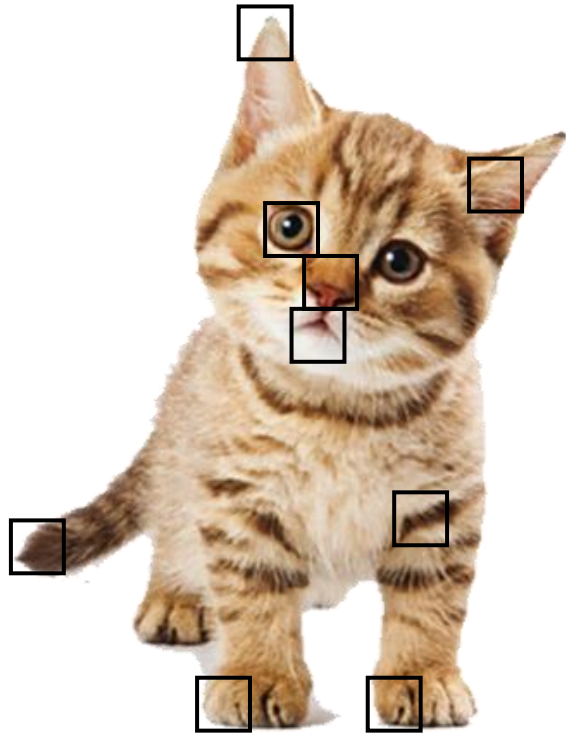
- DNNs **aprendem padrões globais**.
- Elas **ignoram** a **localidade de referência ou espacial** em dados com topologia de grade (e.g., imagens).
  - **Localidade espacial**: em dados com topologia de grade, é a **região** onde **informações próximas** umas das outras **estão relacionadas**.
- São ineficientes computacionalmente e semanticamente (significado de cada região).

# Localidade de referência



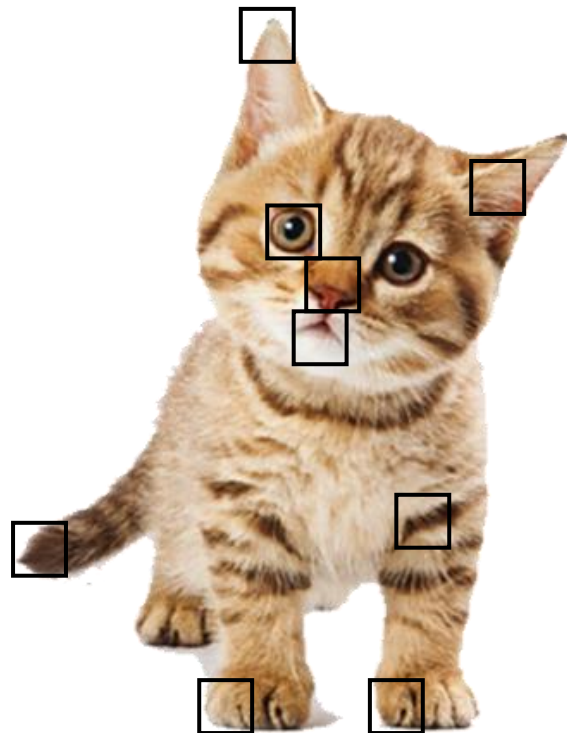
- **Imagens têm uma estrutura espacial** que pode ser explorada por modelos de ML.
- Os *pixels* possuem **posição no espaço**.
- Em uma imagem, *pixels* **vizinhos** tendem a conter informações **relacionadas**.
  - Existem correlações locais entre *pixels* dos olhos, nariz, boca do gato, por exemplo.

# Localidade de referência



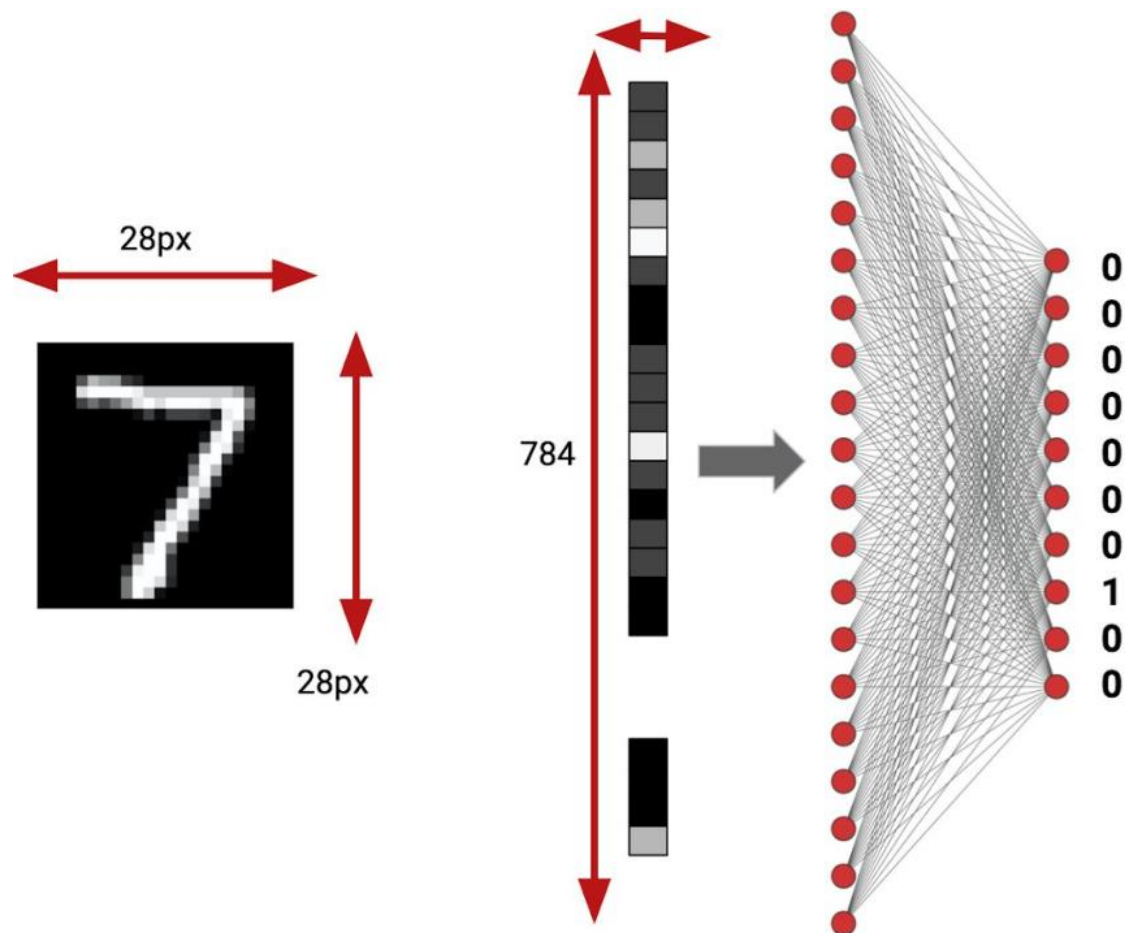
- Estruturas importantes como **bordas, formas e texturas** dependem da **organização espacial**.
- A exploração da **localidade espacial** é **benéfica** para tarefas de **reconhecimento de padrões (estruturas)**.

# DNNs e imagens



- As DNNs **tratam pixels distantes** uns dos outros **da mesma forma que pixels próximos**.
- Por exemplo, se embaralharmos os *pixels* das imagens de entrada de uma DNN treinada para classificar imagens de gatos e cachorros, ela ainda identificará os animais, mesmo as imagens não mais fazendo sentido visual algum.
- Porém, existem alguns problemas ao usarmos DNNs com imagens.

# DNNs e imagens

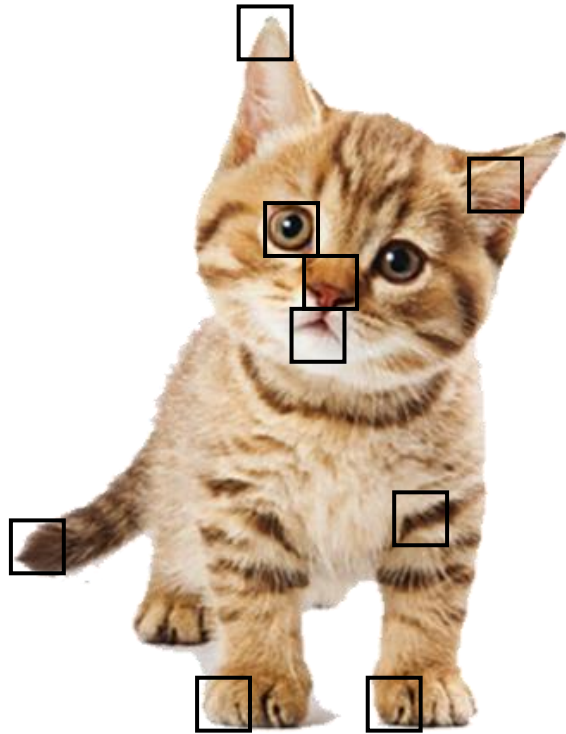


- Redes totalmente conectadas conectam cada *pixel* a todos os nós da próxima camada, gerando um número enorme de pesos, o que resulta em uma **rede muito grande**.
- Exemplo:
  - Uma imagem de 28 x 28 *pixels* gera uma entrada com 784 atributos.
  - Se a primeira camada tiver 100 neurônios serão  $784 \times 100 = 78400$  pesos.
  - E isso apenas na primeira camada.

# DNNs e imagens

- Problemas causados por um modelo grande
  - alto custo computacional
    - treinamento mais lento
    - maior consumo de memória
  - maior risco de *overfitting*
- Além disso, por **desconsiderarem a estrutura espacial** dos *pixels*, seriam necessárias **redes e datasets extremamente grandes** para funcionarem com **problemas de detecção e localização de objetos**, os quais se **baseiam na localização dos pixels**.

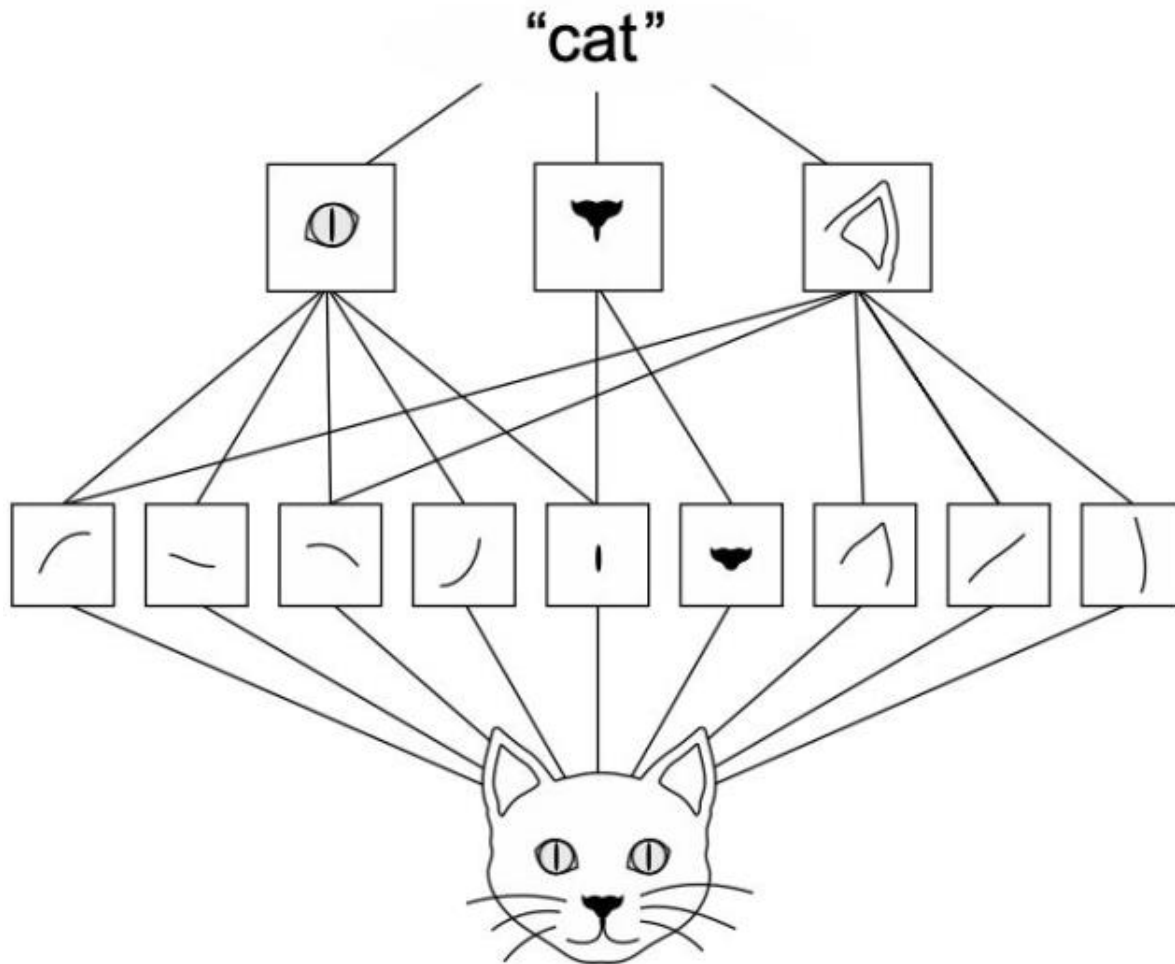
# DNNs e imagens



- Assim, a **conectividade total dos neurônios em uma DNN é um desperdício** para propósitos como o **processamento de imagens**, as quais são
  - **dominadas por padrões espacialmente locais**
  - **e contêm hierarquias.**
    - Características simples se unem para criar características mais complexas.
    - Por exemplo, bordas e cantos formam contornos e texturas, que formam partes do objeto e que por sua vez formam o objeto completo.

Por que usar CNNs?

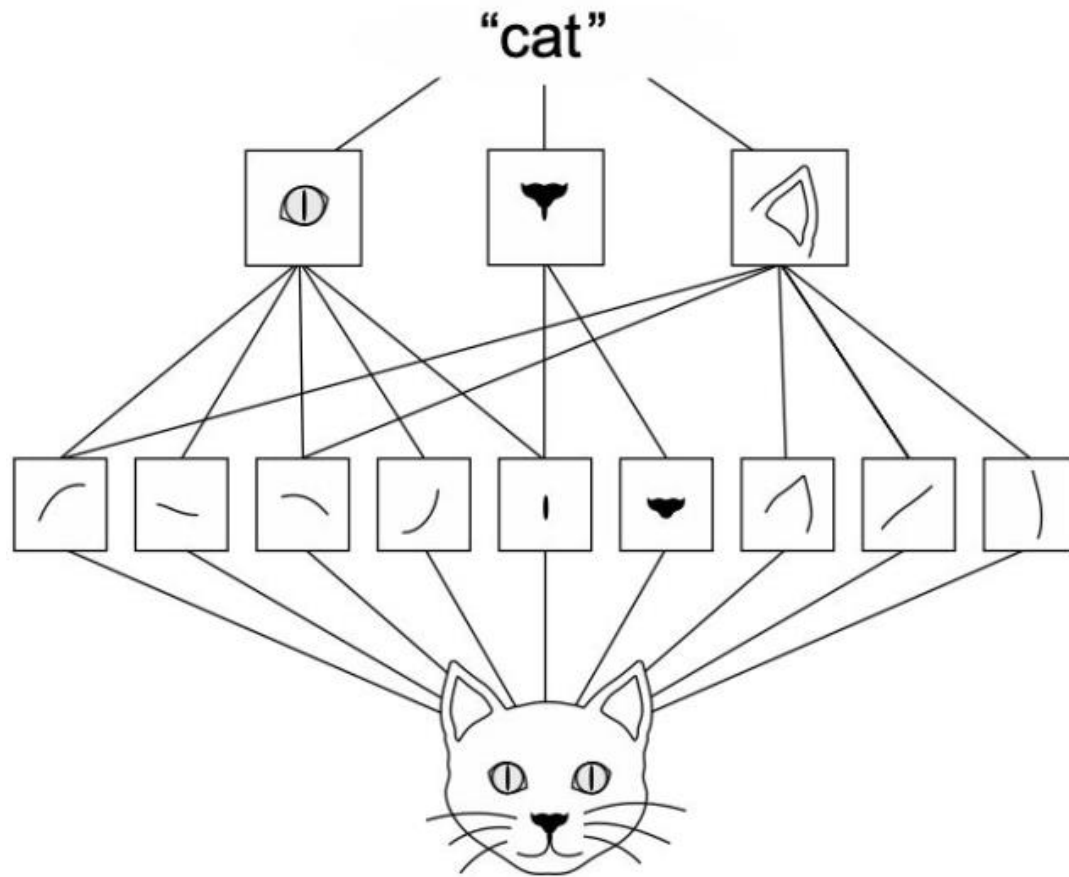
# Redes neurais convolucionais



- CNNs são mais eficientes e exploram as propriedades naturais das imagens:
  - **Localidade espacial**
  - **Estacionariedade**
  - **Composição hierárquica de padrões**

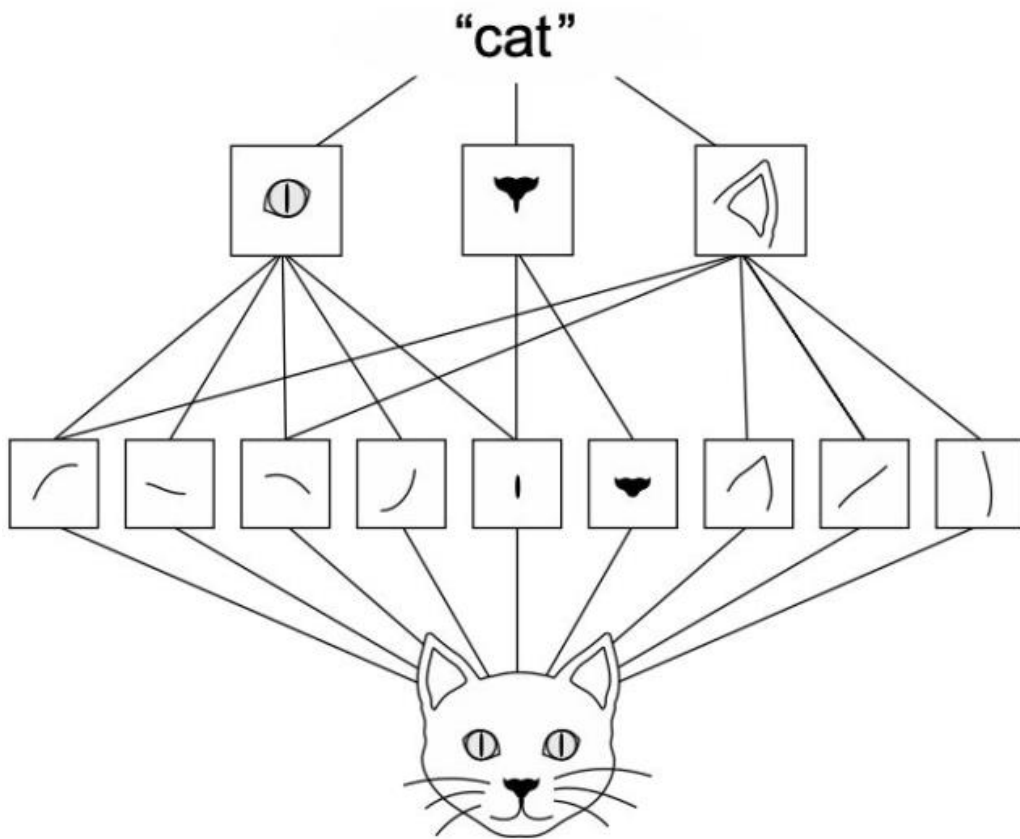
OBS.: CNNs são, em geral, usadas com imagens, mas podem ser aplicadas a outros tipos de dados (e.g., áudio)

# Localidade espacial



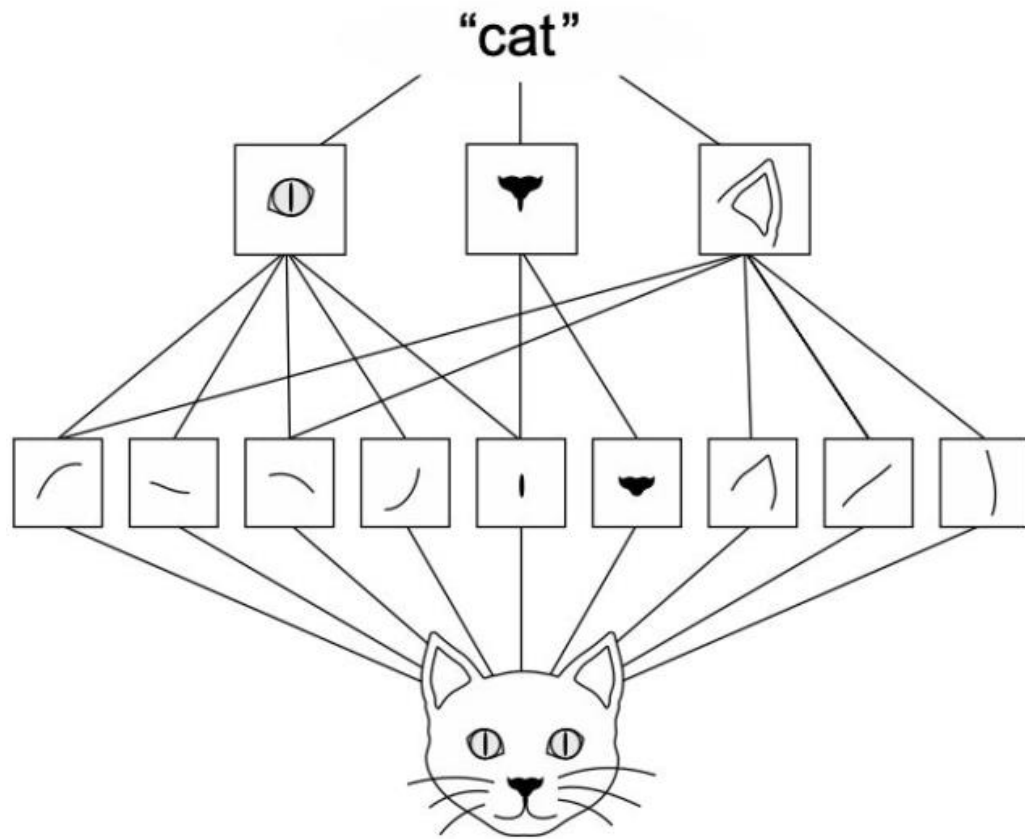
- A propriedade da **localidade espacial** diz que *pixels* próximos tendem a ser **correlacionados**.
- Por exemplo, os *pixels* dos olhos do gato.
- Ou seja, a **maior parte da informação visual** está em **pequenas regiões da imagem**.
- Como as CNNs exploram isso?

# Localidade espacial



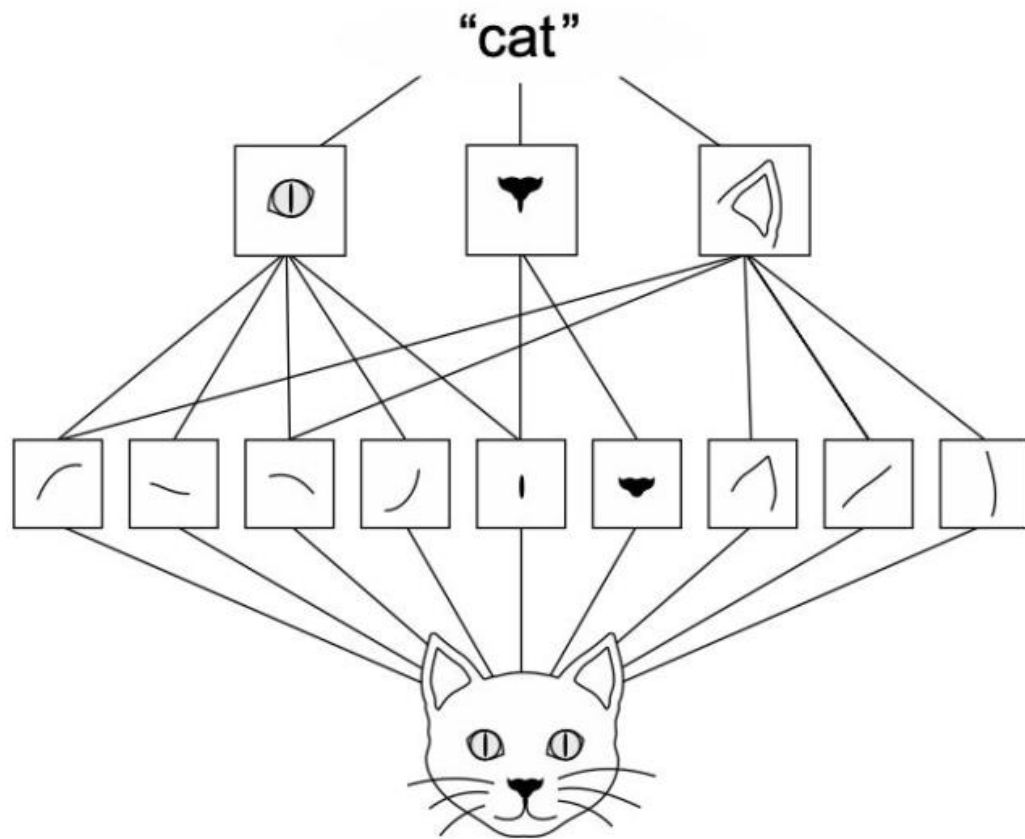
- Em vez de **conectar cada neurônio da CNN a todos os pixels** da imagem, cada neurônio olha apenas para uma **pequena região**, chamada de **campo receptivo**.
- O tamanho do campo receptivo é definido por uma matriz (2D ou 3D) chamada de **filtro de convolução** ou **kernel**.
- Isso permite detectar **padrões locais** como:
  - bordas
  - cantos
  - texturas
  - cores

# Estacionariedade



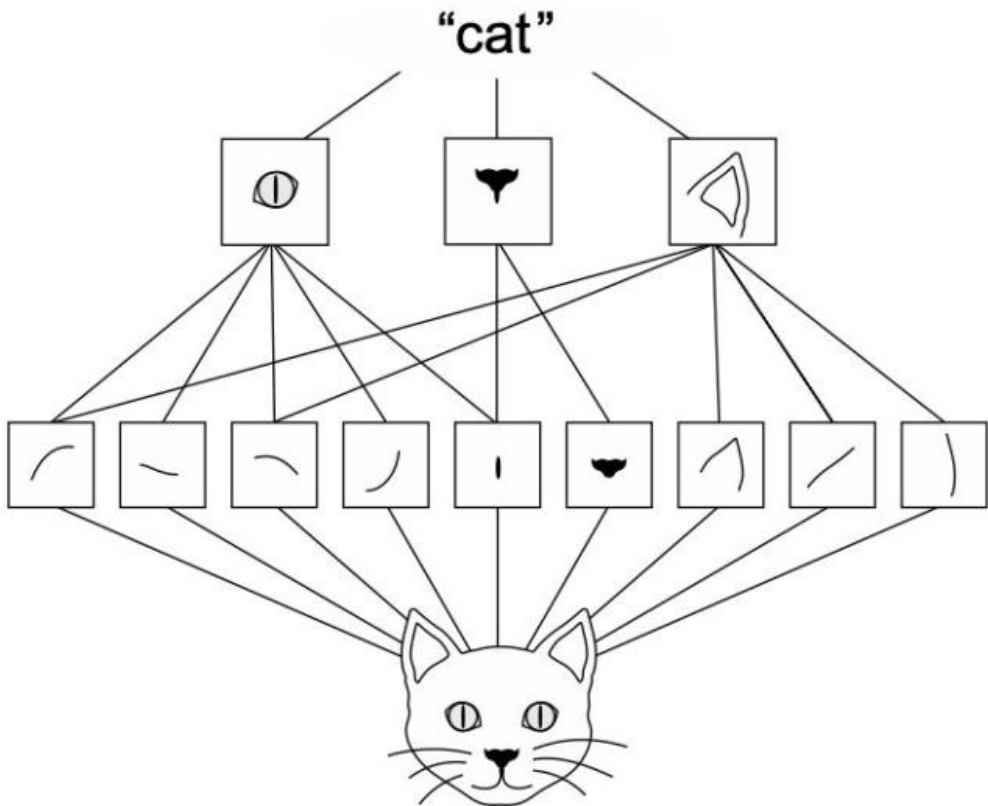
- A estacionariedade significa que o **mesmo padrão pode aparecer em qualquer posição da imagem.**
- Por exemplo:
  - olhos, orelhas e patas do gato podem aparecer em diferentes regiões da imagem
  - uma borda pode aparecer em qualquer lugar
- Como as CNNs exploram isso?

# Estacionariedade



- As **CNNs compartilham os pesos**: o **mesmo filtro** é aplicado em **toda a imagem**.
- Ou seja, um filtro que detecta bordas é o mesmo para todas as posições.
- Isso traz duas **vantagens** enormes:
  - muito menos parâmetros.
  - melhora a generalização (modelo menor, menos risco de *overfitting*).
  - capacidade de detectar o mesmo padrão em qualquer posição

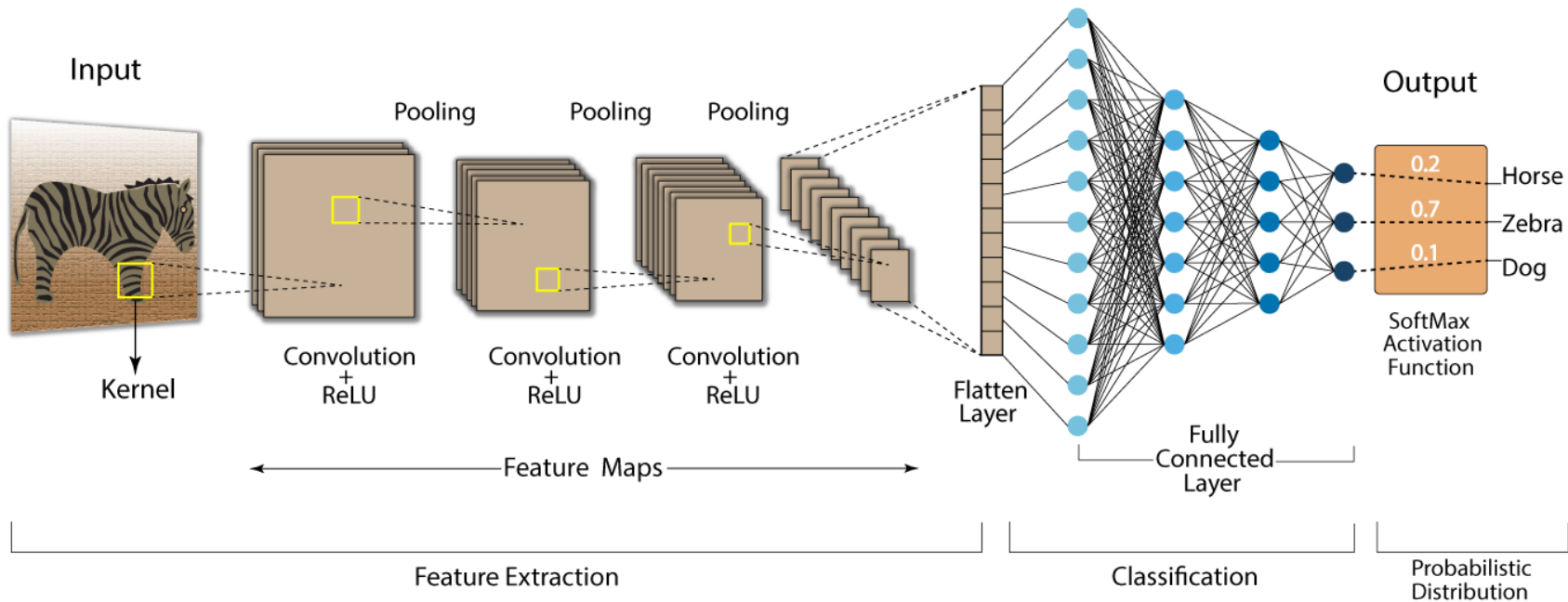
# Composição hierárquica de padrões



- Imagens possuem uma **estrutura hierárquica**.
- Padrões simples formam padrões mais complexos:
  - **Simples**: contorno da orelha, borda do nariz e do olho
  - **Intermediários**: curva da orelha, forma triangular do nariz
  - **Partes menores**: olho , nariz, bigode, pata, cauda
  - **Partes maiores**: rosto, cabeça, corpo
  - **Objeto completo**
- As **camadas** de uma CNN **aprendem a hierarquia** de padrões da imagem.
  - bordas → contornos → partes → objeto

Como é a arquitetura de uma  
CNN?

# Arquitetura geral de uma CNN



- Uma CNN para classificação de imagens é composta por:

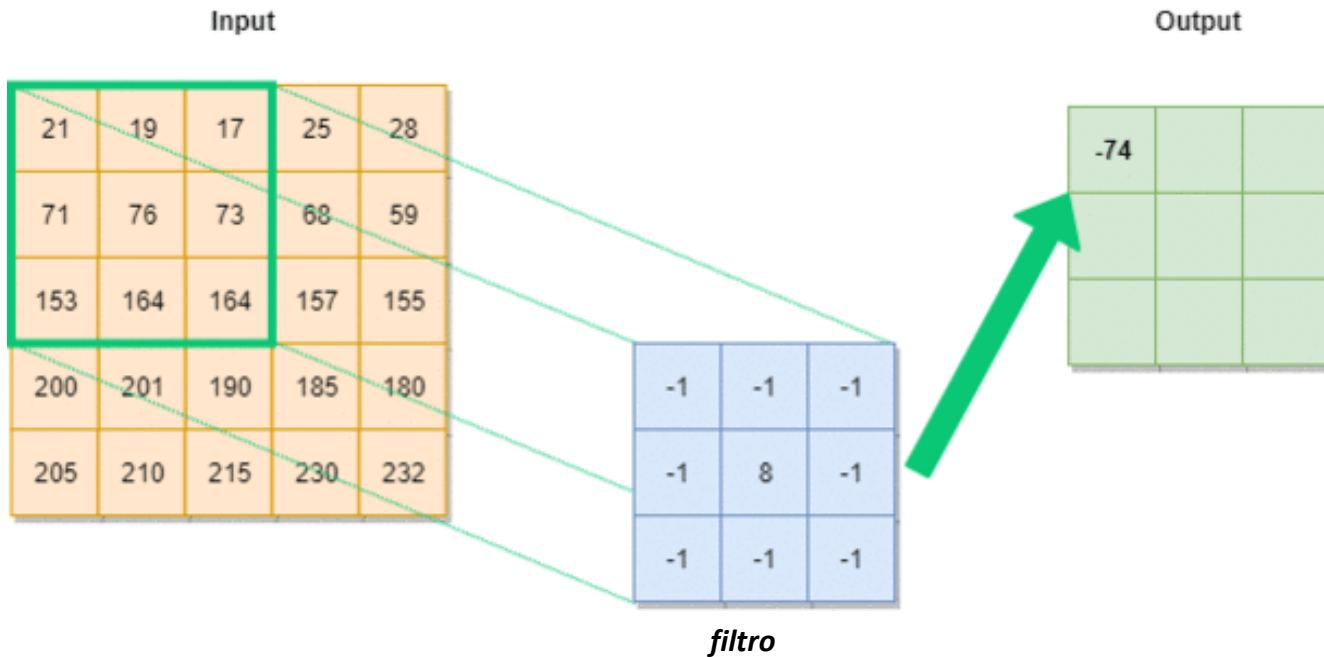
- Uma entrada (imagem)
- Um ou mais grupos de
  - Camadas de convolução
    - ✓ Operações de convolução
    - ✓ Funções de ativação
  - Camadas de *pooling*
- Uma camada *flatten*
- Uma ou mais camadas totalmente conectadas
- Uma função de ativação *softmax*

# Imagens



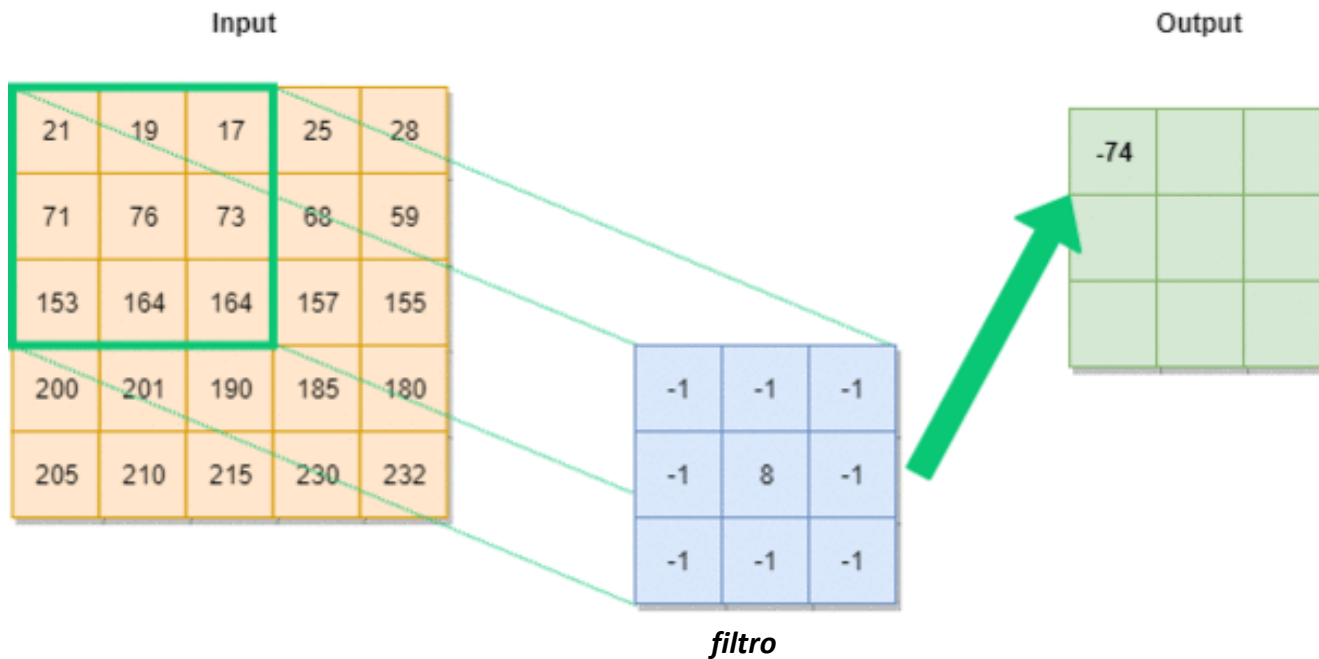
- Podemos agrupar as imagens em dois tipos: **coloridas** e em **tons de cinza**.
- Em imagens coloridas, cada *pixel* é **formado por três valores**: vermelho (R), verde (G) e azul (B).
- Imagens coloridas são **tridimensionais** (3D):
  - altura × largura × 3 canais
  - Cada canal indicada a quantidade de cada cor.
- Imagens em tons de cinza são **bidimensionais**:
  - altura × largura
  - Cada *pixel* carrega apenas informação de **intensidade luminosa**

# Camada de convolução



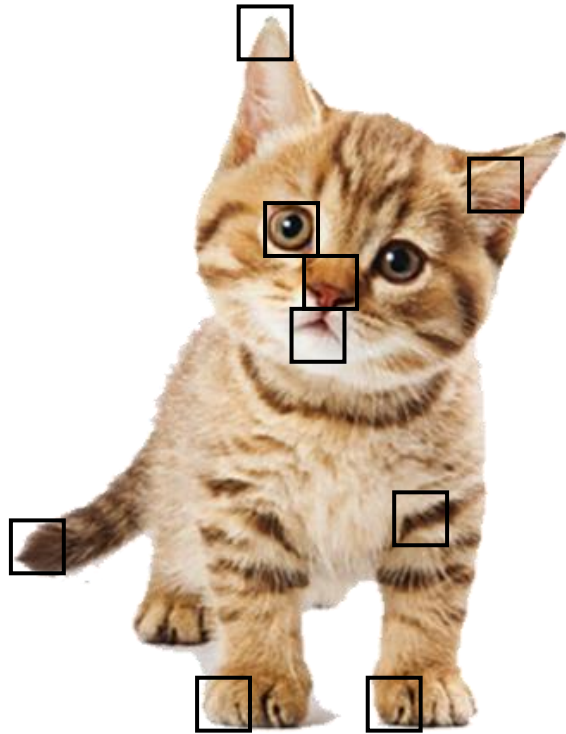
- A camada de convolução é responsável por **extrair características da imagem**.
- Ela faz isso **deslizando filtros** sobre a imagem.
  - **Filtros**: matrizes 2D ou 3D.
- Esse filtro:
  - é multiplicado com valores locais da imagem (campo receptivo)
  - soma os resultados,
  - e gera uma saída escalar.

# Camada de convolução



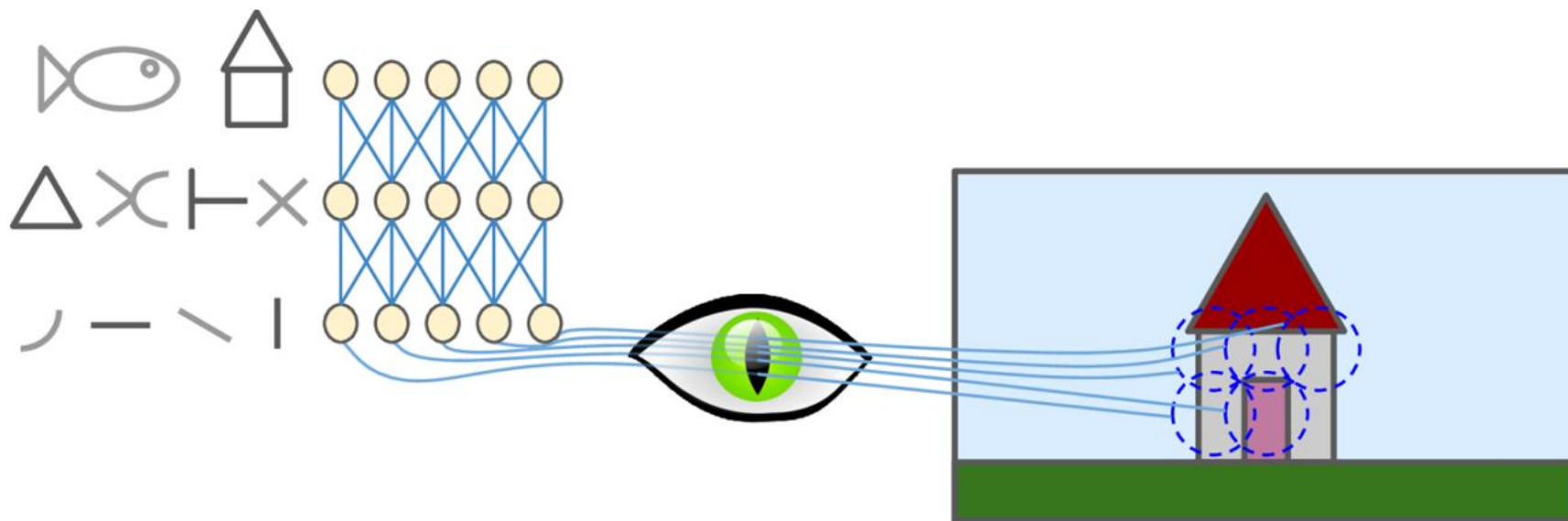
- Após o filtro, aplica-se uma **função de ativação**.
- Esse processo é repetido por toda a imagem.
- O resultado é um **mapa de características** (*feature map*) **detectadas**.
- Ou seja, de padrões detectados.

# Filtros de convolução ou *kernels*



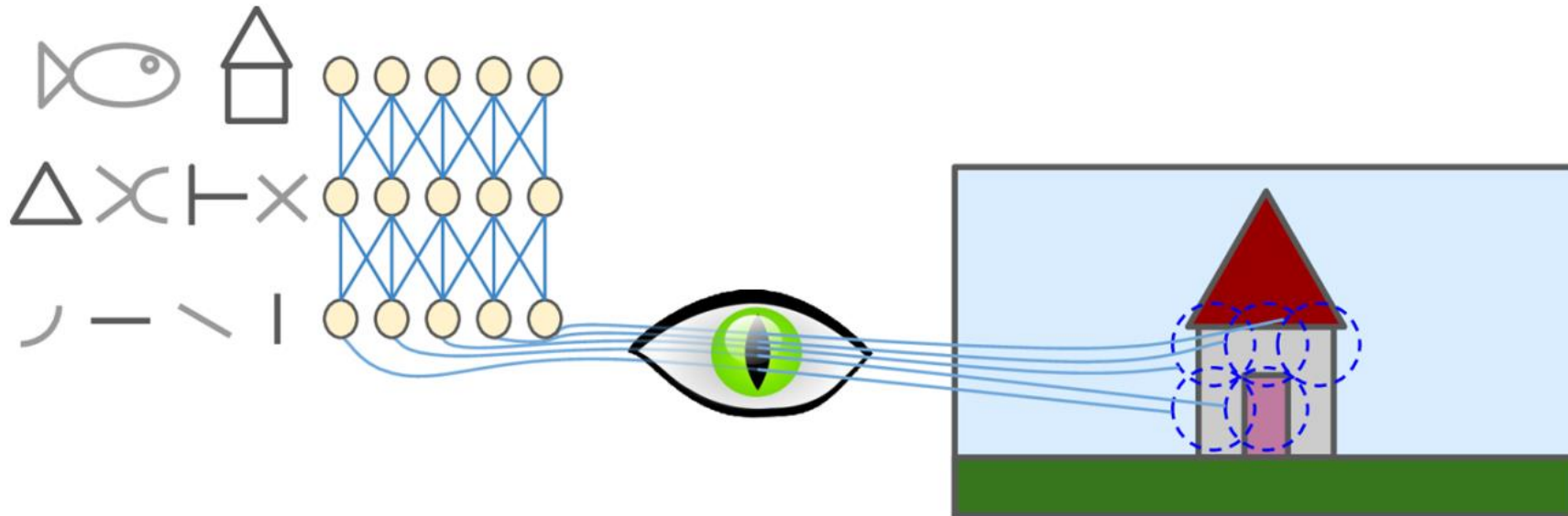
- Os **filtros** são **treinados para detectar características (padrões) dos objetos**.
  - Os filtros têm pesos que são aprendidos.
- Essa ideia de usar **filtros** que **detectam características** que fazem um **objeto ser diferente de outro** é **baseada** em como os neurônios do **córtex visual** **funcionam**.

# Neurônios biológicos



- Os **neurônios biológicos** no córtex visual **respondem a padrões específicos** em **pequenas regiões do campo visual** chamadas **campos receptivos**.
- À medida que o sinal visual percorre as camadas do cérebro, os **neurônios respondem a padrões mais complexos em campos receptivos maiores**.

# Neurônios biológicos



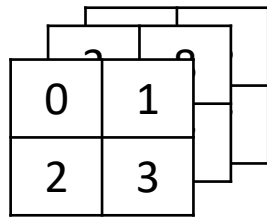
- Alguns **neurônios biológicos** reagem a **linhas horizontais**, enquanto **outros**, reagem a **linhas com orientações diferentes**.
- Outros têm **campos receptivos maiores** e reagem a **padrões mais complexos** que são **combinações de padrões mais simples**.

# Filtros de convolução ou *kernels*

Kernel 2D

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

Kernel 3D



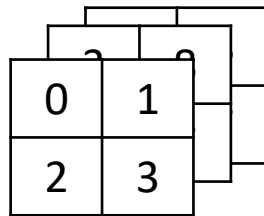
- Um *kernel* é uma **matriz** (ou tensor) responsável por **detectar características específicas em uma imagem**.
- Ele percorre uma imagem realizando **operações convolução** entre seus valores e os dos *pixels* na região da imagem correspondente ao seu **campo receptivo**.
- Para imagens **coloridas**, eles têm **3 dimensões** e **2 dimensões** para imagens em **tons de cinza**.

# Filtros de convolução ou *kernels*

Kernel 2D

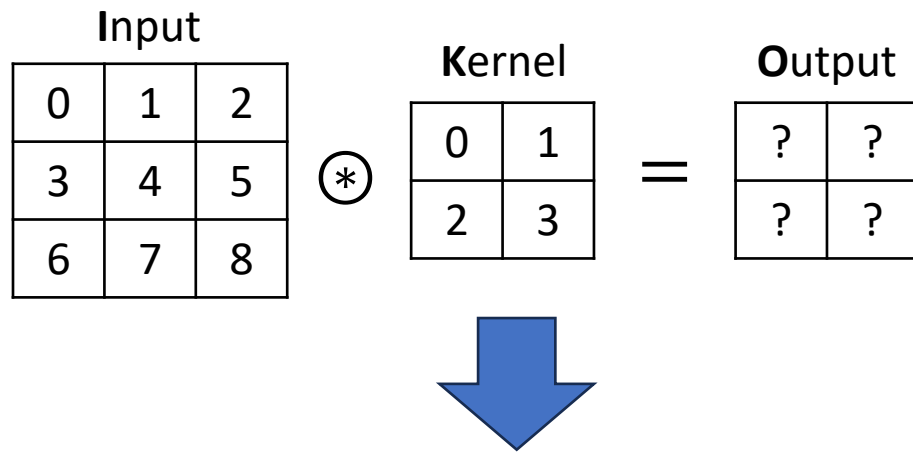
|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

Kernel 3D



- Cada **kernel** detecta um tipo particular de **característica**, como bordas, texturas, padrões ou partes específicas de objetos.
- Os **kernels** aprendem a **detectar** as **características mais relevantes** para a tarefa em mãos.
  - Eles **aprendem**, a partir do **conjunto de treinamento**, os valores dos **elementos da matriz**, necessários para **detectar uma determinada característica**.

# Operação de convolução com 1 canal



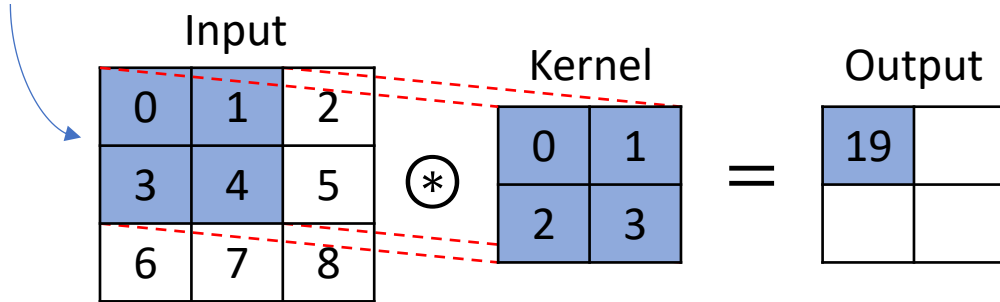
$$O(i, j) = \sum_m \sum_n I(i + m, j + n) K(m, n)$$

$I$  : input  
 $K$ : kernel  
 $O$ : output

- Vamos ignorar múltiplos canais por enquanto e ver como uma operação de convolução funciona com dados bidimensionais.
- O símbolo  $\circledast$  representa a operação de “convolução”.
- A saída da operação de convolução é um **mapa de características**.
- O operação é representada pela equação ao lado.

# Operação de convolução com 1 canal

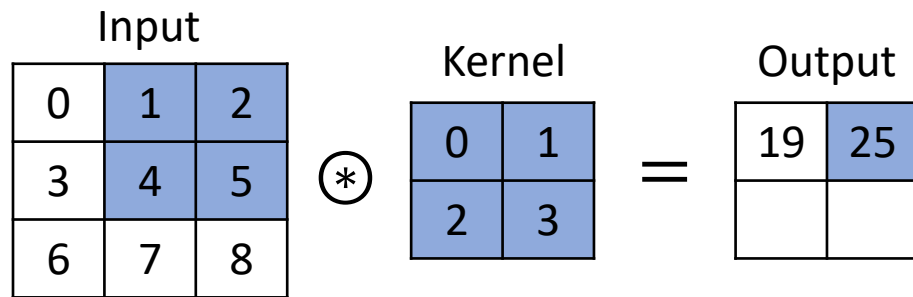
Janela de convolução  
ou campo receptivo



- Ao calcular a convolução, começamos com a **janela de convolução** no **canto superior esquerdo** do tensor de entrada.

$$\begin{aligned} O(i, j) &= \sum_m \sum_n I(i + m, j + n) K(m, n) \\ &= 0 * 0 + 1 * 1 + 3 * 2 + 4 * 3 \\ &= 19 \end{aligned}$$

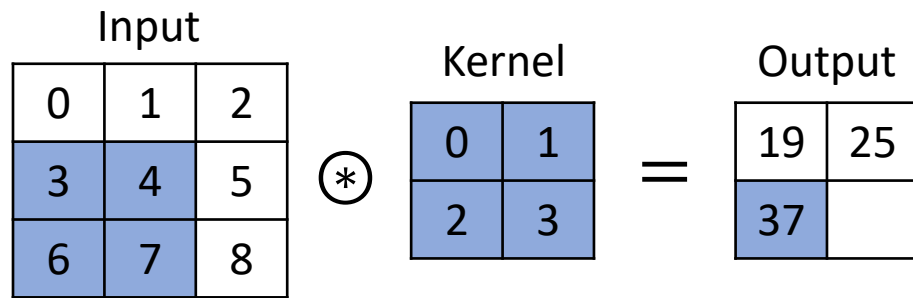
# Operação de convolução com 1 canal



- Em seguida, deslizamos a janela, por exemplo, um elemento para a direita.
- O deslocamento é definido pelo *stride*, o qual veremos a seguir.

$$\begin{aligned} O(i, j) &= \sum_m \sum_n I(i + m, j + n) K(m, n) \\ &= 1 * 0 + 2 * 1 + 4 * 2 + 5 * 3 \\ &= 25 \end{aligned}$$

# Operação de convolução com 1 canal



- Ao chegar-se ao final das colunas do tensor de entrada, volta-se ao seu início, deslizando a janela, por exemplo, um elemento para baixo, ou seja, uma linha.

$$\begin{aligned} O(i, j) &= \sum_m \sum_n I(i + m, j + n) K(m, n) \\ &= 3 * 0 + 4 * 1 + 6 * 2 + 7 * 3 \\ &= 37 \end{aligned}$$

# Operação de convolução com 1 canal

| Input |   |   |
|-------|---|---|
| 0     | 1 | 2 |
| 3     | 4 | 5 |
| 6     | 7 | 8 |

 $\otimes$ 

| Kernel |   |
|--------|---|
| 0      | 1 |
| 2      | 3 |

 $=$ 

| Output |    |
|--------|----|
| 19     | 25 |
| 37     | 43 |

- Em seguida, deslizamos a janela um elemento para a direita.
- Esse processo se repete até que a janela de convolução tenha percorrido todo o tensor de entrada.

$$\begin{aligned} O(i, j) &= \sum_m \sum_n I(i + m, j + n) K(m, n) \\ &= 4 * 0 + 5 * 1 + 7 * 2 + 8 * 3 \\ &= 43 \end{aligned}$$

# Mapa de características

| Input |   |   |
|-------|---|---|
| 0     | 1 | 2 |
| 3     | 4 | 5 |
| 6     | 7 | 8 |

 $\otimes$ 

| Kernel |   |
|--------|---|
| 0      | 1 |
| 2      | 3 |

 $=$ 

| Output |    |
|--------|----|
| 19     | 25 |
| 37     | 43 |

- Lembrando que o **objetivo** dos *kernels* é **extrair (ou identificar) características**.
- Portanto, o resultado da operação de convolução é chamado de **mapa de características ou de ativação**.
- É chamado assim, pois **fornece as ativações desse filtro em cada posição espacial** do tensor (imagem).
- O mapa mostra **onde** e com que **intensidade** uma característica específica (detectada pelo filtro) está presente no tensor de entrada.

# Stride

- O **stride** controla o deslizamento do *kernel* sobre o tensor.
- É o número de *pixels* que o *kernel* avança a cada aplicação da convolução.
- No exemplo anterior, o **stride** era de 1 *pixel*.
- Para que serve sua variação?
  - Reduzir o tamanho da *feature map* (*downsampling*)
    - Aumentar o *stride* **reduz a resolução da saída**.
  - Reduzir o custo computacional.
    - Aumentar o *stride* **reduz a quantidade de cálculos**.
  - Controlar o nível de detalhe.
    - *Stride* pequeno captura detalhes menores: detecta padrões finos.
      - ✓ Porém, é mais caro computacionalmente.
    - *Stride* grande captura detalhes maiores: detecta padrões mais globais.

# Stride

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 0 | 1 |
| 2 | 3 | 4 | 5 |

 $\otimes$ 

|        |   |
|--------|---|
| Kernel |   |
| 0      | 1 |
| 2      | 3 |

 = 

|        |  |
|--------|--|
| Output |  |
| 24     |  |
|        |  |



Input

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 0 | 1 |
| 2 | 3 | 4 | 5 |

 $\otimes$ 

|        |   |
|--------|---|
| Kernel |   |
| 0      | 1 |
| 2      | 3 |

 = 

|        |    |
|--------|----|
| Output |    |
| 24     | 36 |
|        |    |



Input

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 0 | 1 |
| 2 | 3 | 4 | 5 |

 $\otimes$ 

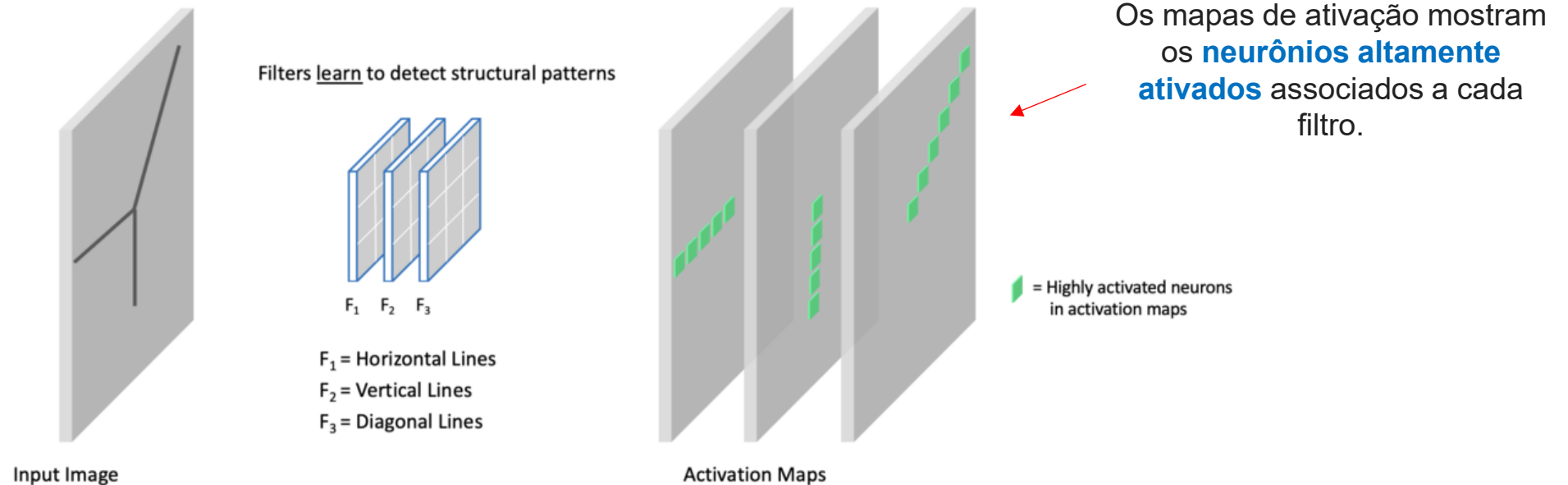
|        |   |
|--------|---|
| Kernel |   |
| 0      | 1 |
| 2      | 3 |

 = 

|        |    |
|--------|----|
| Output |    |
| 24     | 36 |
| 22     |    |

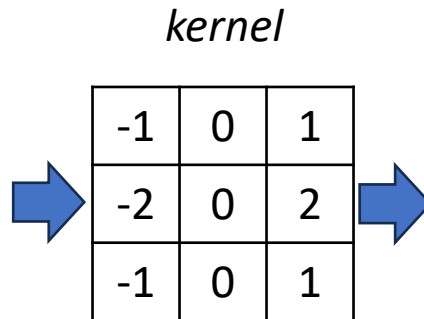
- Portanto, o *stride* controla o equilíbrio entre **precisão vs. eficiência**.
- No exemplo ao lado, o *stride* é de 2 para deslocamentos ao longo das colunas e linhas.
- Porém, ele pode ser diferente para deslocamentos ao longo das linhas e colunas.

# Filtros detectam padrões



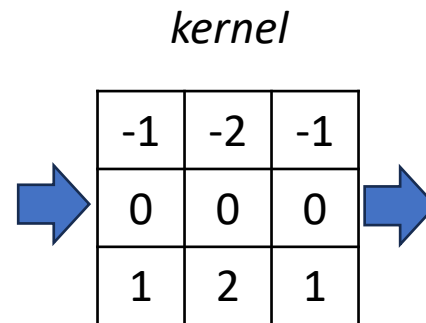
- Na figura, a imagem de entrada apresenta alguns padrões lineares (bordas).
- Temos uma camada convolucional com três filtros.
- Cada filtro aprende a detectar diferentes padrões: linhas horizontais, linhas verticais e linhas diagonais.

# Aplicando *kernels* a imagens



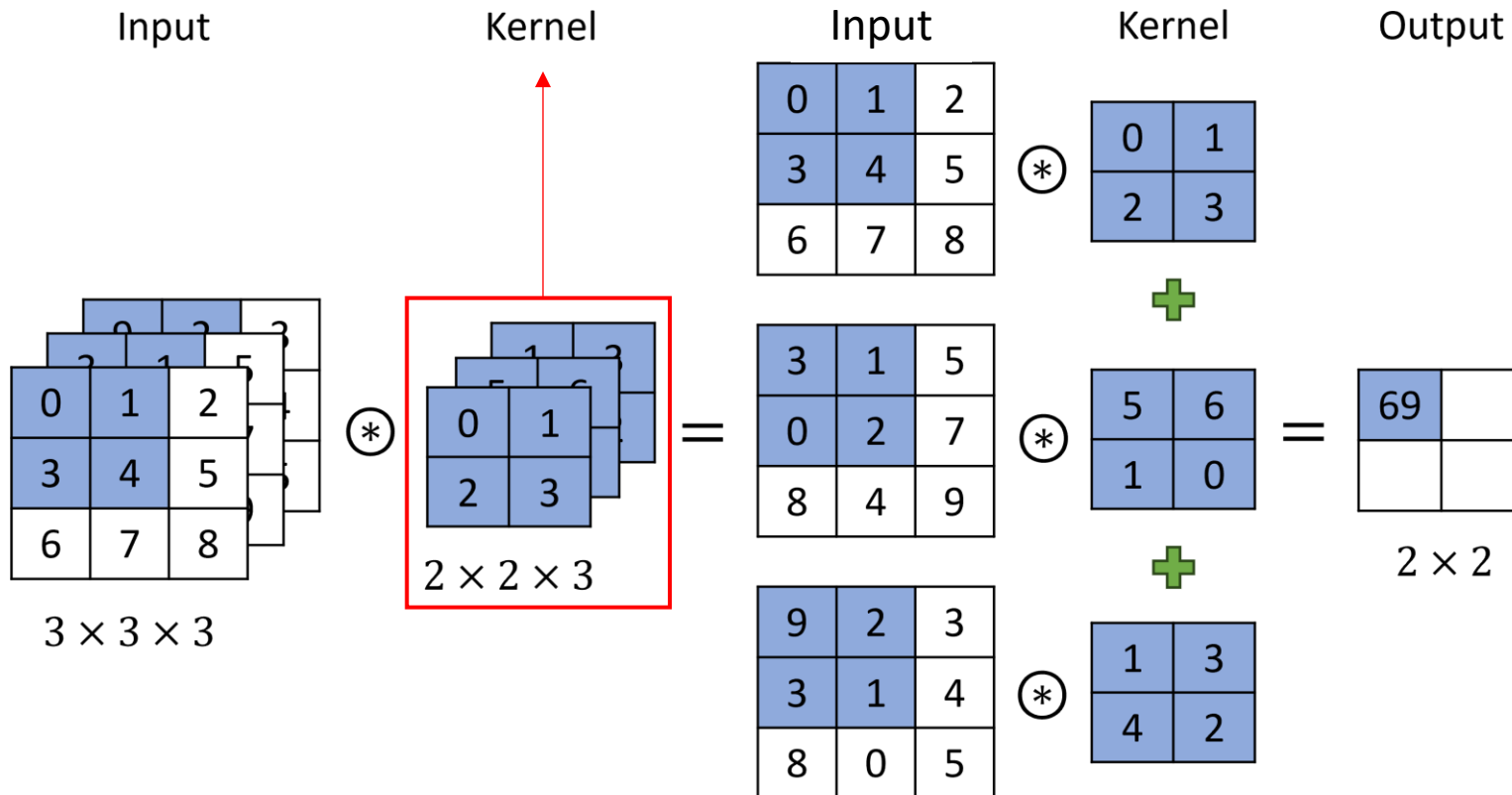
- Considerem a imagem em tons de cinza à esquerda.
- Se aplicarmos o *kernel* mostrado, obteremos os resultados à direita.
- Ele **realça muito as linhas verticais e escurece todo o resto**.
- Portanto, podemos dizer que este *kernel* é um **detector de linhas verticais**.

# Aplicando *kernels* a imagens



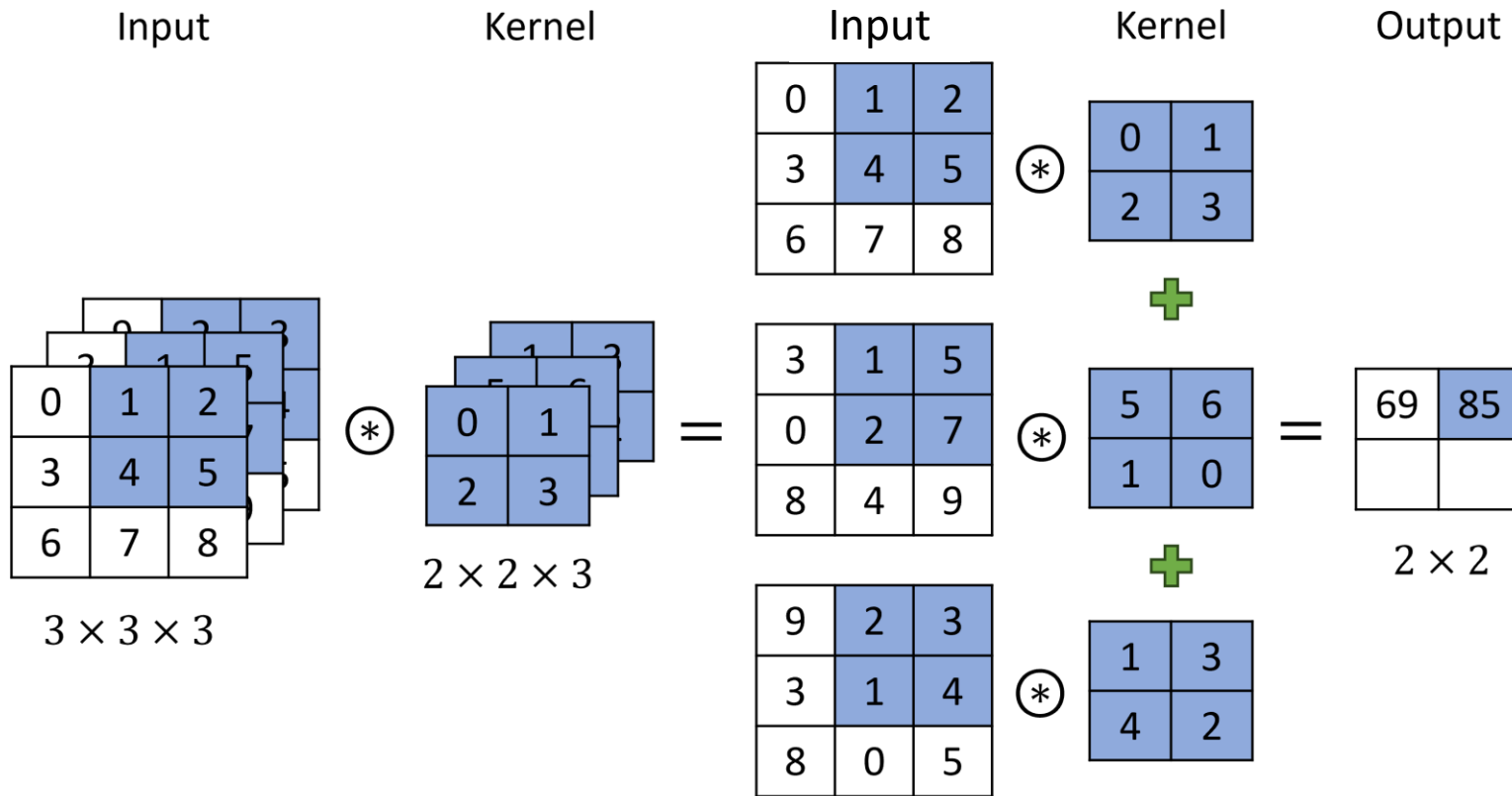
- Já o *kernel* acima, **detecta linhas horizontais**, escurecendo quase tudo na imagem que não seja uma linha horizontal.
- Ao aplicar *kernels* como esses, podemos **remover quase tudo, exceto uma característica distinta**.
- Esse é o processo chamado de **extração de características**.
  - Processo que determina as partes mais importantes de uma imagem.

# Operação de convolução com 3 canais

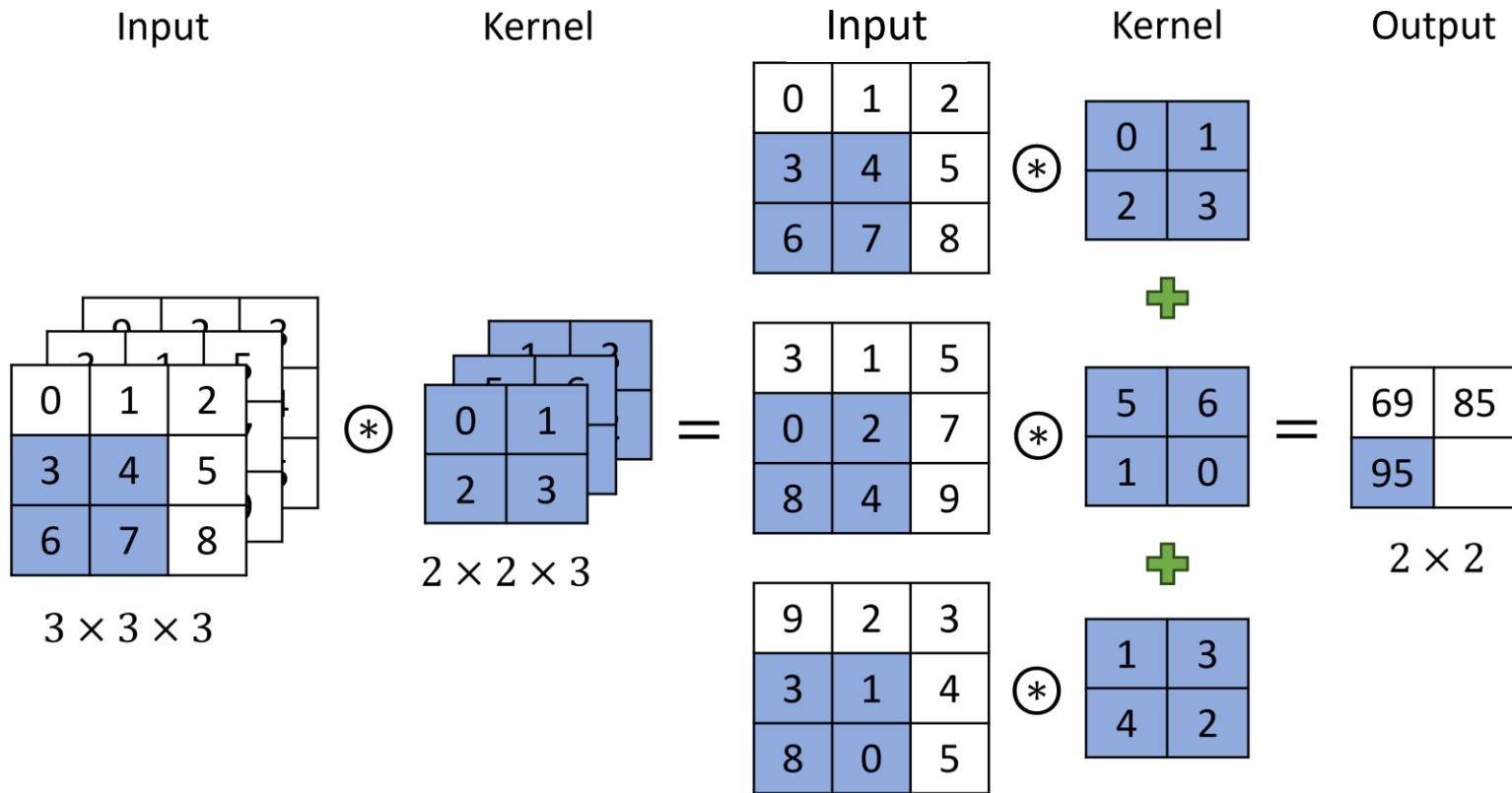


- Em geral, se a imagem tem 3 dimensões, o **kernel** também terá 3 dimensões.
- Para entender a operação, podemos **dividi-la em 3 operações de convolução separadas** que têm seus resultados somados ao final para gerar a saída.
- Usando um *stride* = 1, temos.

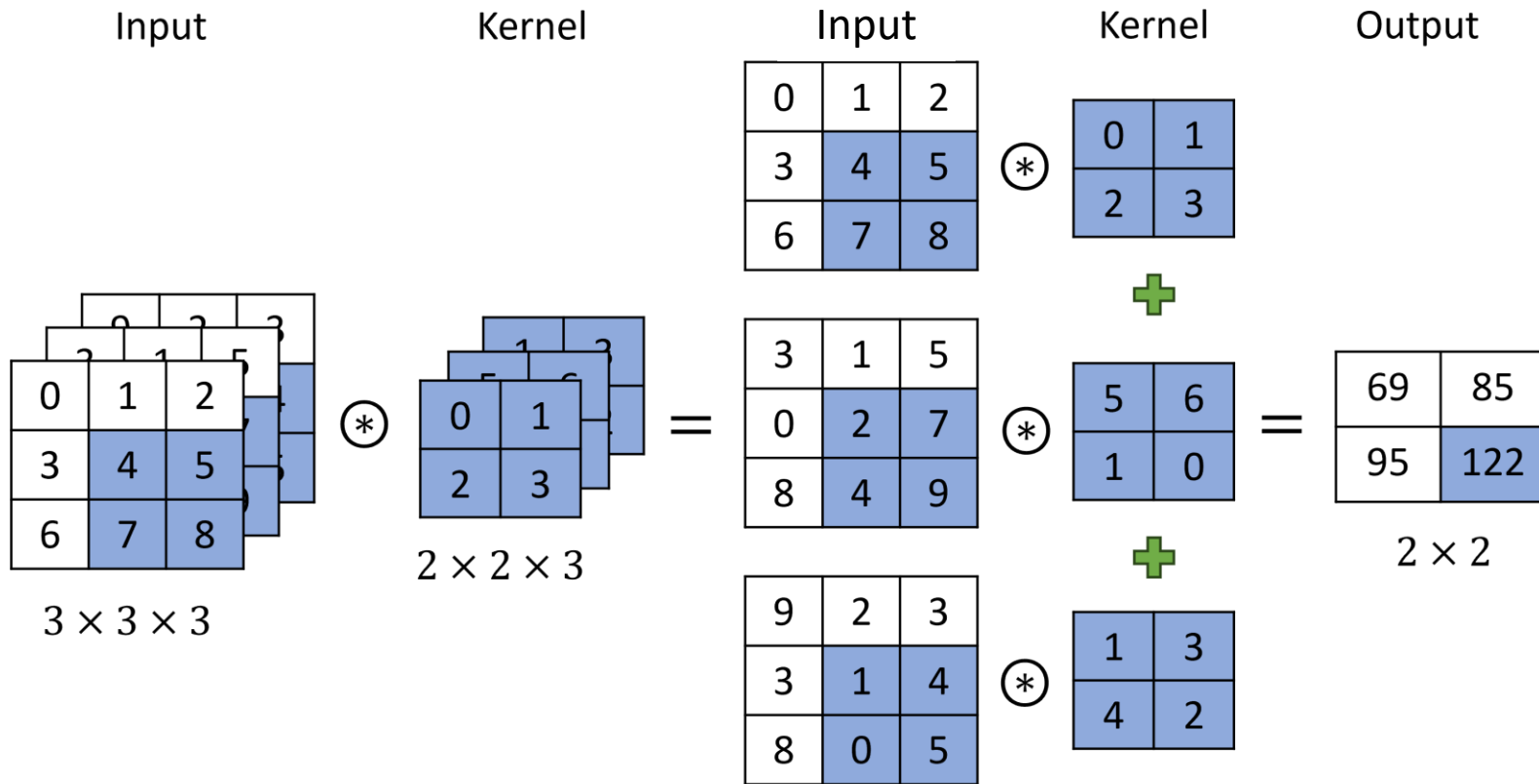
# Operação de convolução com 3 canais



# Operação de convolução com 3 canais



# Operação de convolução com 3 canais



# Função de ativação

Feature map,  $M$

|      |      |
|------|------|
| -1.5 | 10   |
| 22   | -3.7 |



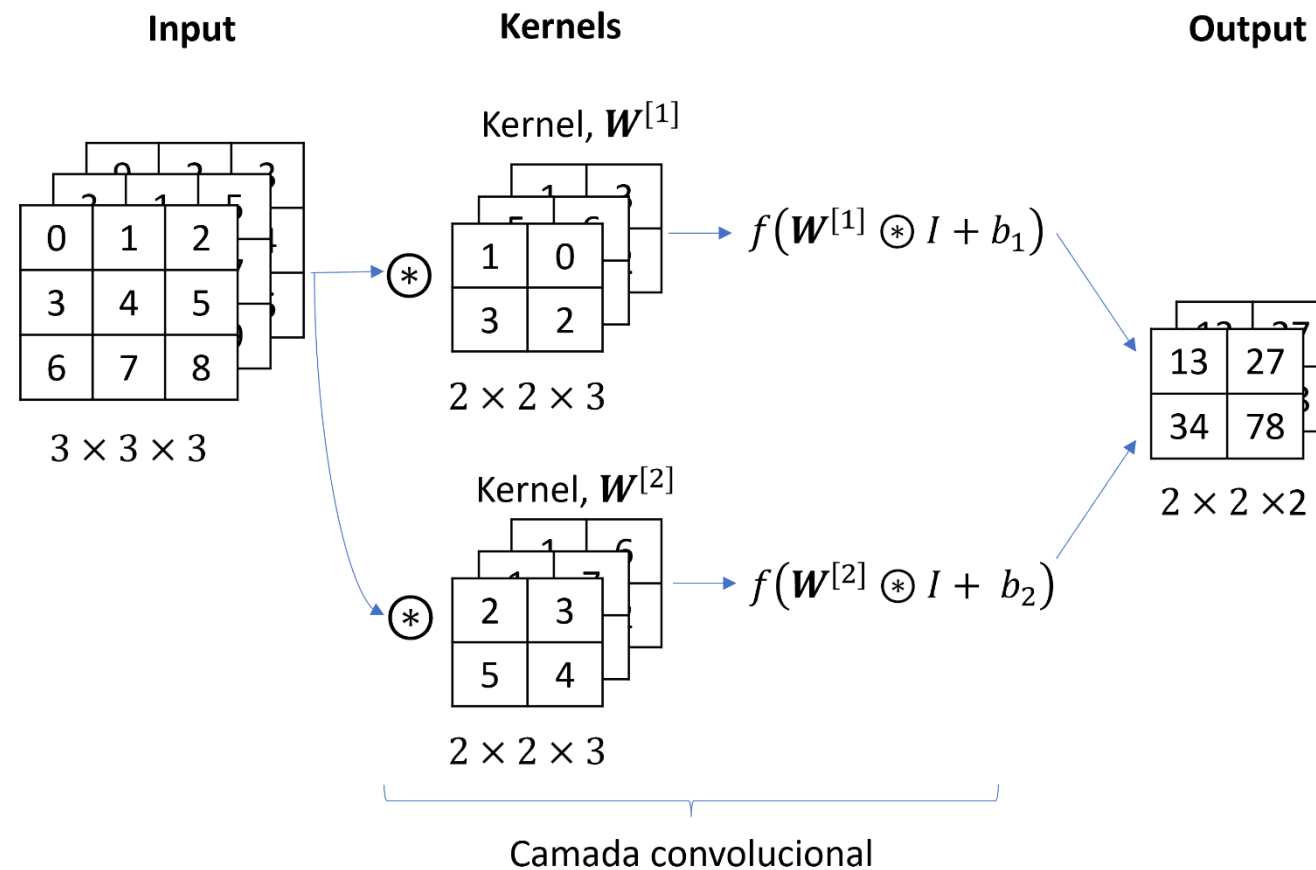
$$f(M) = \max(0, M)$$



|    |    |
|----|----|
| 0  | 10 |
| 22 | 0  |

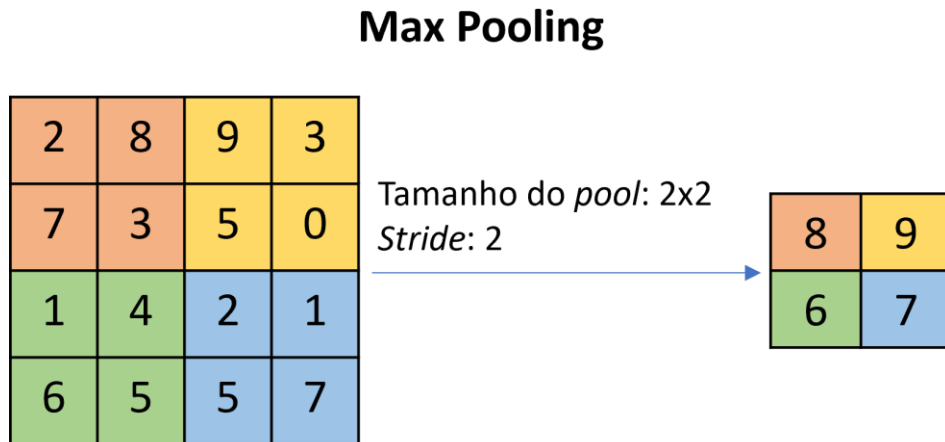
- A função de ativação introduz não linearidade.
- Ela é aplicada diretamente ao mapa de características **elemento a elemento**.
- Isso significa que cada valor do mapa é transformado individualmente pela função de ativação.
- Em geral, para evitar o desaparecimento do gradiente, se usa funções de ativação do tipo ReLU ou suas variantes.

# Kernels diferentes para características diferentes



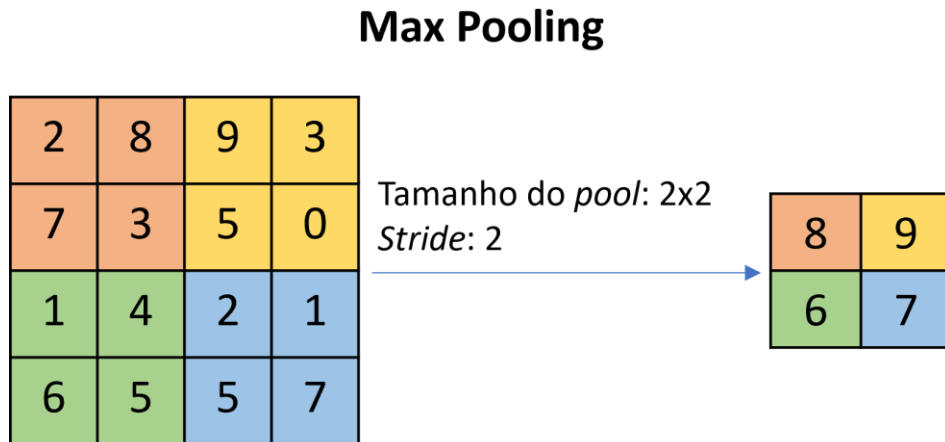
- Em geral, cada **camada convolucional possui vários kernels**.
- Cada **kernel detecta uma característica diferente**.
- O resultado da convolução com cada **kernel** tem um valor de *bias* somado a ele e o resultado é passado pela função de ativação,  $f(\cdot)$ .
- A saída da camada é o resultado do **empilhamento de várias matrizes**.

# Camada de *Pooling* (ou subamostragem)



- Aplicada **após uma camada de convolução**.
- Ela **subamostra** sua entrada: reduz o tamanho espacial do mapa de características.
- O **objetivo** da subamostragem é
  - **reduzir a carga computacional,**
  - **o uso de memória**
  - **e o número de parâmetros.**
- Isso limita o risco de **sobreajuste**.

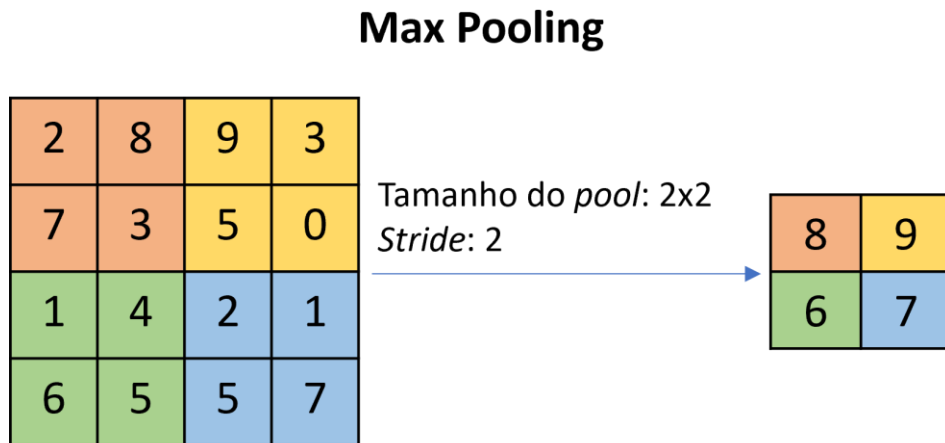
# Camada de *Pooling* (ou subamostragem)



- Além disso, ela **ajuda a tornar a rede mais robusta a pequenas mudanças na posição das características.**
- Ela cria invariância local.
  - Pequenos deslocamentos das características não afetam muito o resultado.
  - Foca na presença do padrão, não na localização.
- O padrão pode:
  - estar deslocado levemente
  - ter pequenas deformações
  - ter ruído

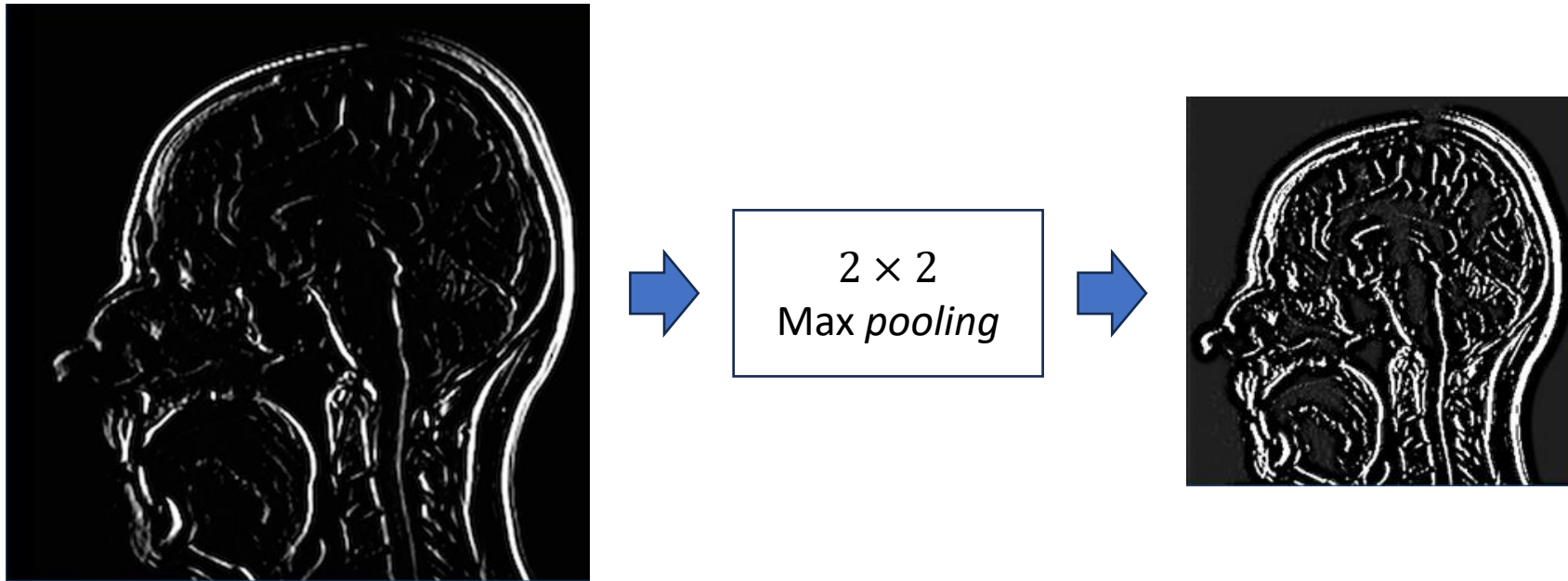
e mesmo assim a CNN continua o reconhecendo.

# Max pooling



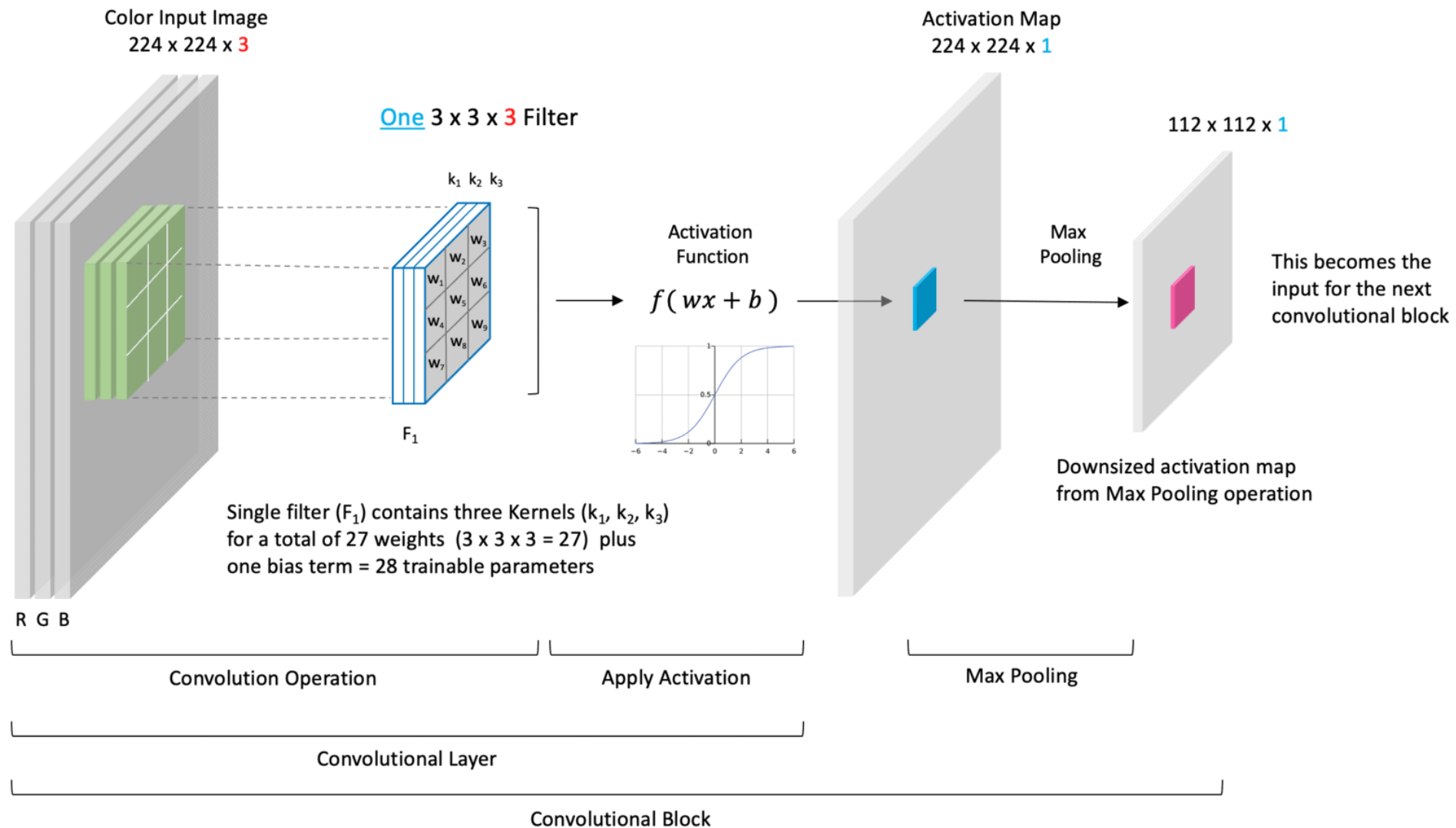
- A subamostragem mais usada é o *max pooling*, que aplica a **operação  $\max(.)$**  a cada mapa de características de uma camada de convolução.
- Somente o **valor máximo em cada campo receptivo do pool passa para a próxima camada**, enquanto os outros são descartados.
- Ele mantém apenas **as características mais importantes** (mais fortes), ao mesmo tempo que **descarta as menos relevantes ou ruidosas**.

# *Max pooling*



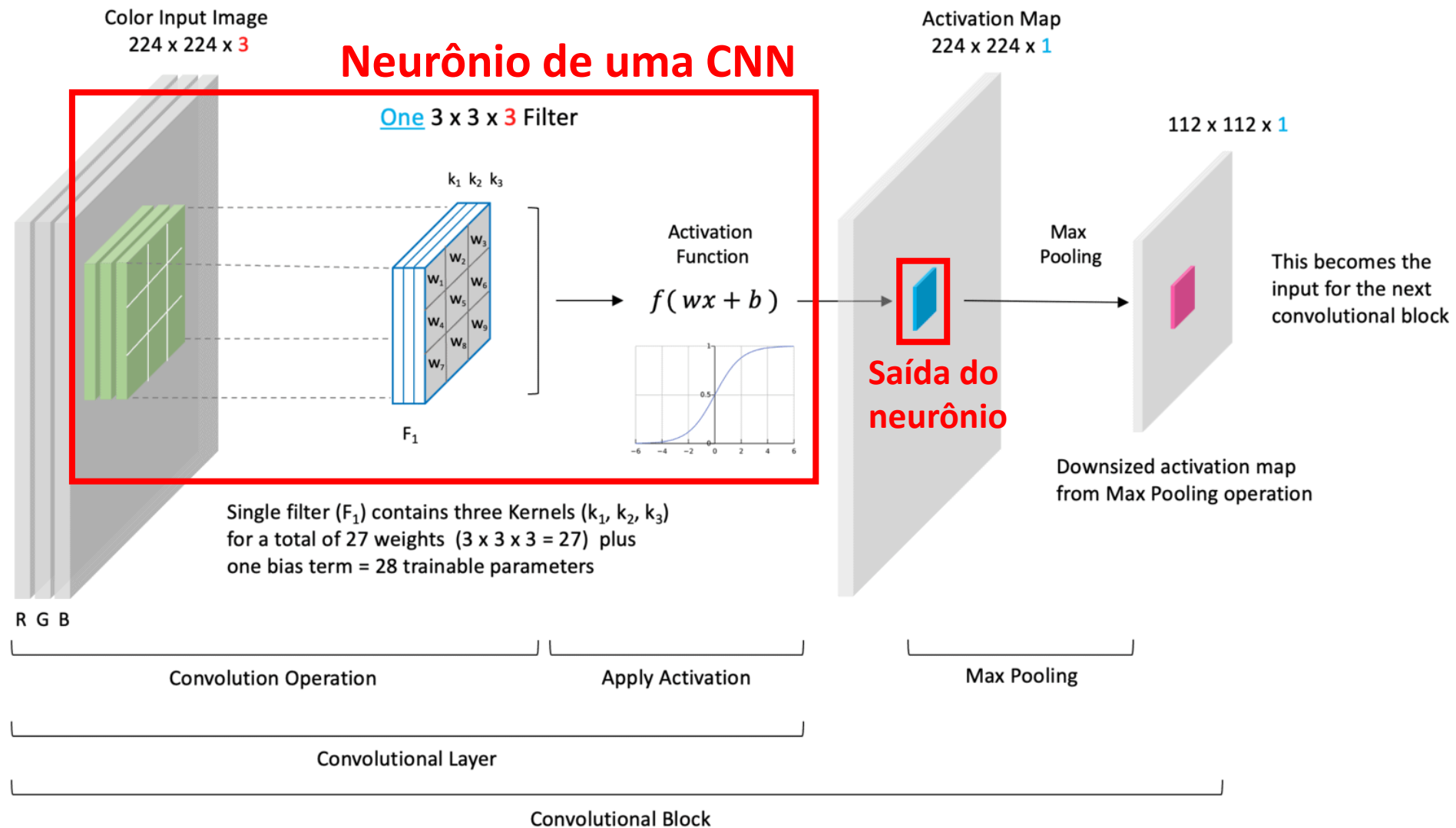
- Na figura, o *max pooling* diminui a resolução, deixa as bordas do cérebro bem destacadas e descarta detalhes finos, como pequenos objetos, bordas muito finas, pequenas variações de intensidade, etc.

# Bloco de convolução



**OBS.:** assume-se uma configuração de preenchimento que mantém o tamanho espacial dos dados em camadas convolucionais.

# Bloco de convolução



# Padding ou preenchimento

| Input |   |   |
|-------|---|---|
| 0     | 1 | 2 |
| 3     | 4 | 5 |
| 6     | 7 | 8 |

 $\otimes$ 

| Kernel |   |
|--------|---|
| 0      | 1 |
| 2      | 3 |

 $=$ 

| Output |    |
|--------|----|
| 19     | 25 |
| 37     | 43 |

$3 \times 3$                        $2 \times 2$                        $2 \times 2$

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

$n_{out}$  : dimensão da saída

$n_{in}$  : dimensão da entrada

$k$  : dimensão do kernel

$p$  : quantidade camadas de *padding*

$s$  : tamanho do *stride*

$\lfloor x \rfloor$  : função piso retorna o maior inteiro menor ou igual a  $x$ .

- Depois de aplicar muitas convoluções sucessivas, as **imagens tendem** a se **tornarem** consideravelmente **menores** do que as da entrada.
- Por exemplo, se temos uma imagem de entrada com  $240 \times 240$  *pixels*, após dez camadas de convolução  $5 \times 5$  com *stride* igual a 1, ela é reduzida para  $200 \times 200$  *pixels*.
- Isso reduz a imagem em  $\approx 17\%$ .

# Padding ou preenchimento

| Input |   |   |
|-------|---|---|
| 0     | 1 | 2 |
| 3     | 4 | 5 |
| 6     | 7 | 8 |

 $\otimes$ 

| Kernel |   |
|--------|---|
| 0      | 1 |
| 2      | 3 |

 $=$ 

| Output |    |
|--------|----|
| 19     | 25 |
| 37     | 43 |

$3 \times 3$                        $2 \times 2$                        $2 \times 2$

$$n_{out} = \left\lfloor \frac{n_{in} + 2p - k}{s} \right\rfloor + 1$$

$n_{out}$  : dimensão da saída

$n_{in}$  : dimensão da entrada

$k$  : dimensão do kernel

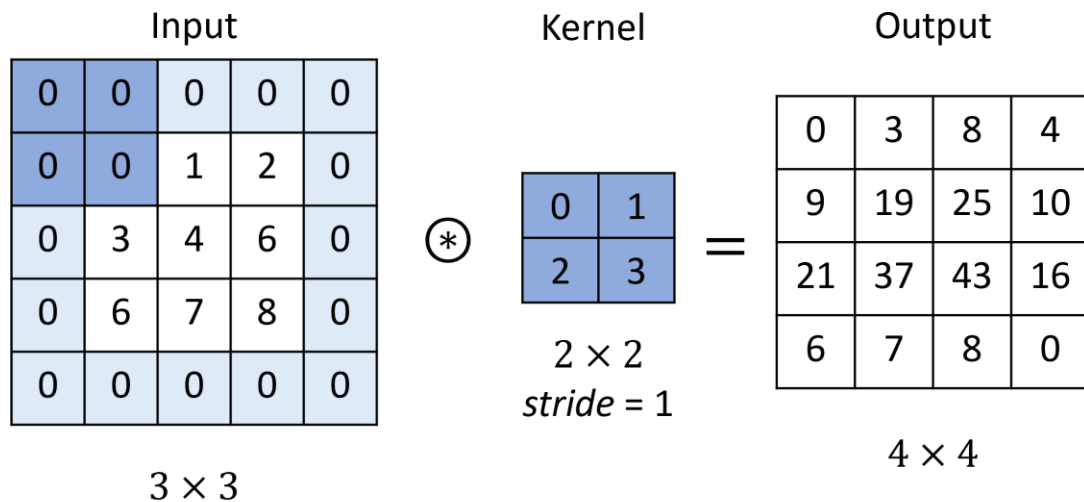
$p$  : quantidade camadas de *padding*

$s$  : tamanho do *stride*

$\lfloor x \rfloor$  : função piso retorna o maior inteiro menor ou igual a  $x$ .

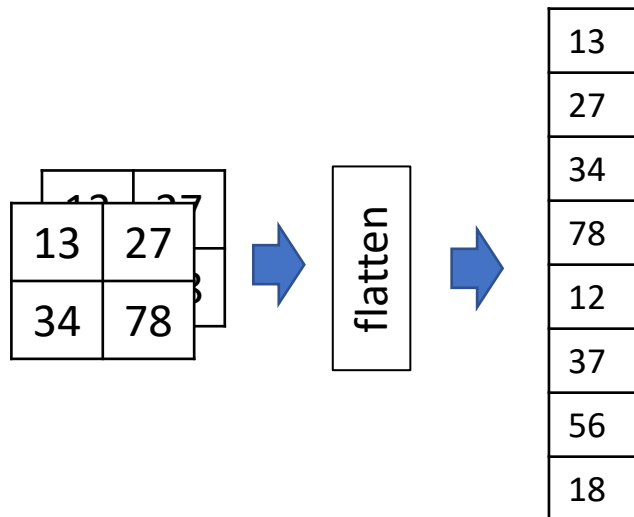
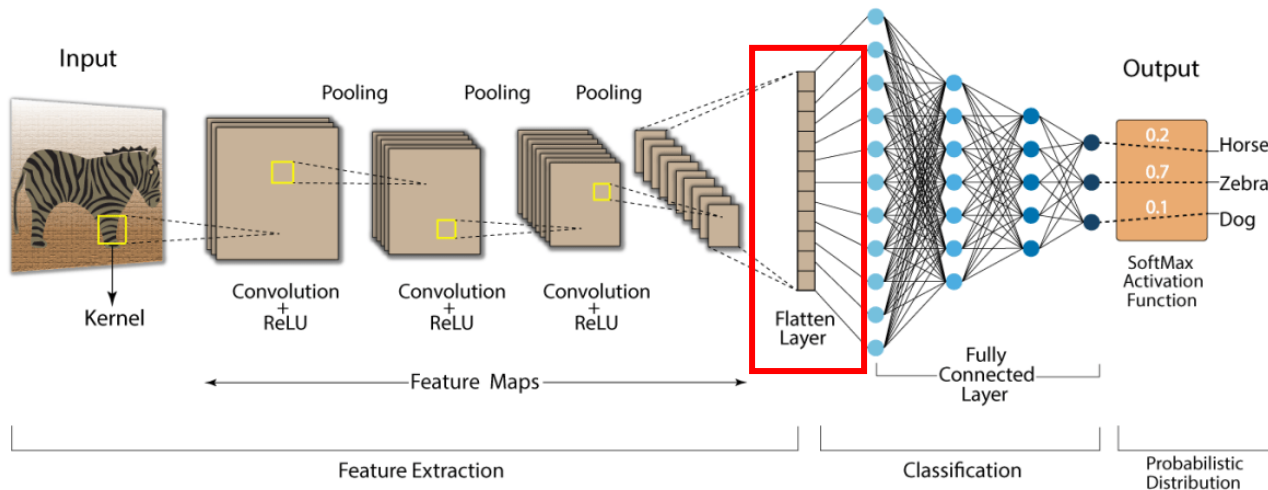
- Além da redução, a convolução faz com que qualquer **informação interessante nas bordas da imagem seja subutilizada**.
- Os *pixels* das bordas de uma imagem são **subutilizados**, i.e., participam de menos convoluções que os *pixels* centrais.
- O *padding* garante que todos os *pixels* contribuam mais igualmente para a extração de características.

# Padding ou preenchimento



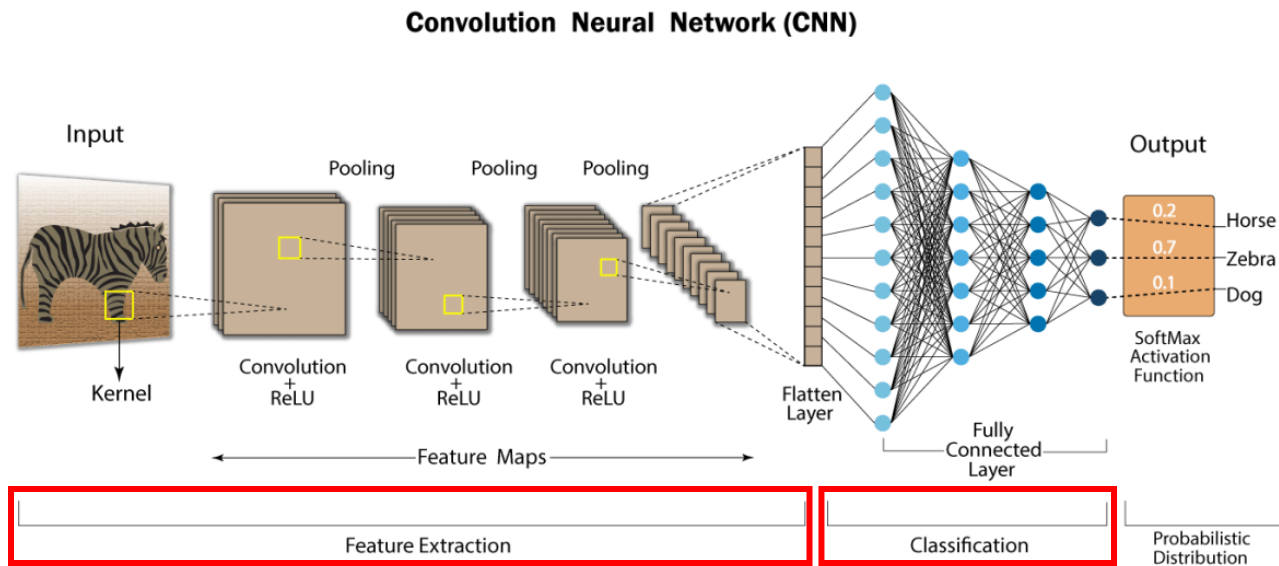
- O **padding** é usado para **controlar o tamanho dos mapas de características** após uma camada convolucional.
- **Pixels de preenchimento são adicionados às bordas da imagem**, aumentando assim seu tamanho efetivo.
  - Normalmente, os **pixels** são feitos iguais a **zero**.
- É comum aplicar **padding** nas **camadas iniciais para preservar informações da borda**, enquanto **camadas finais não o aplicam** para reduzir a dimensionalidade.

# Camada de *flatten*



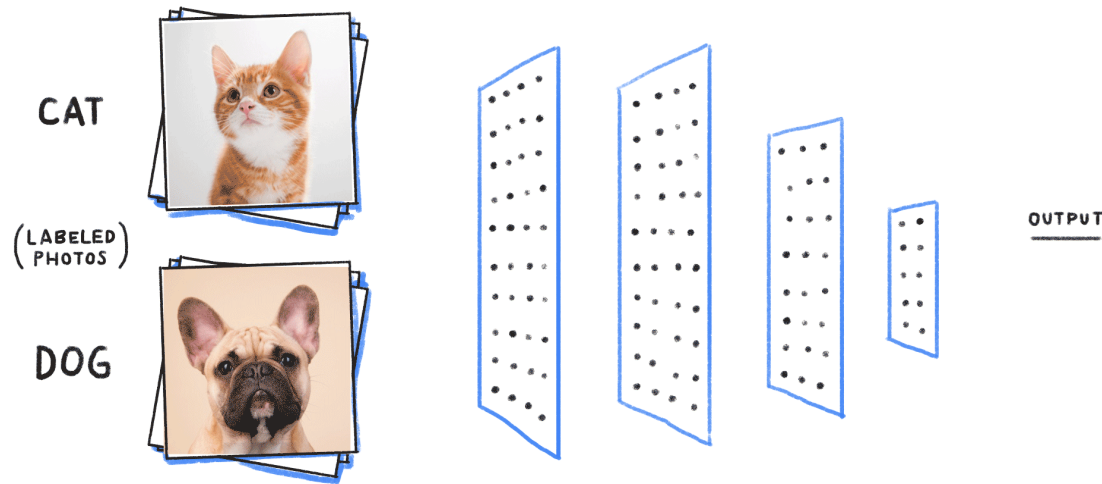
- Na saída da etapa de extração de características temos vários mapas de características, formando um volume 3D de dados.
- Porém, as **camadas densas** trabalham com **vetores 1D**.
- Assim, precisamos transformar os mapas de características em vetores 1D.
- Ela não possui parâmetros treináveis.

# Camadas densas



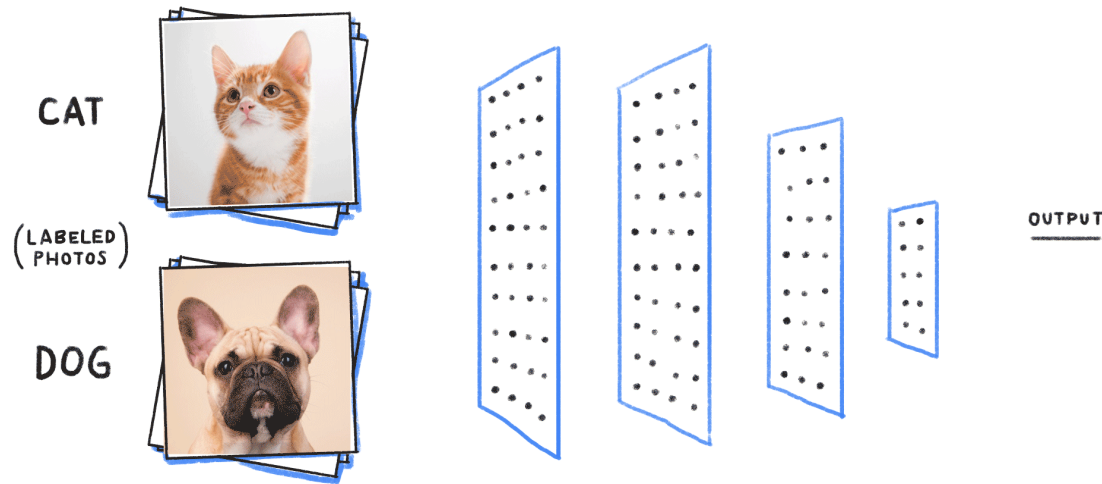
- Depois das camadas convolucionais, as CNN apresentam uma DNN, com uma ou mais camadas densas.
- Elas recebem o vetor 1D de características extraídas pelas camadas convolucionais e as transformam em uma predição final (seja para tarefas de classificação ou de regressão).

# Camadas densas



- Em classificação, elas usam as características para decidir a classe.
- Por exemplo, se as características de um gato aparecem juntas → é um gato.
- Ou seja, ela aprende **relações entre características**.
- Por que usar DNNs no final?
  - Convoluções são locais (olham pedaços da imagem).
  - DNNs olham tudo ao mesmo tempo.

# Camadas densas



- As funções de ativação da camada densa de saída mais usadas são:
  - *Softmax*, para classificação multiclass.
  - Logística, para classificação binária e multi-rótulo.
  - Identidade, para regressão.

# Treinamento de CNNs

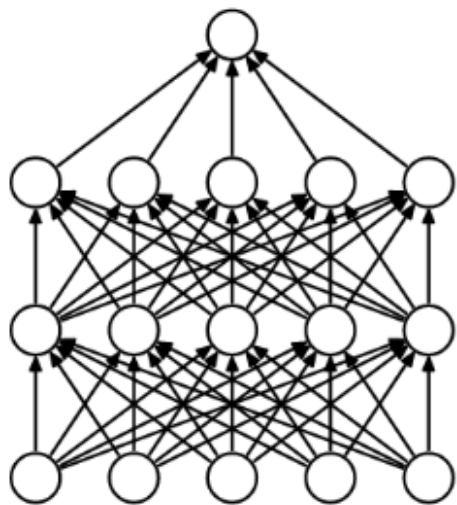
- Assim como outras redes neurais, CNNs são treinadas usando-se o algoritmo da retropropagação do erro.
- O objetivo é aprender os pesos dos filtros e das camadas densas.
- Os gradientes são calculados para os:
  - pesos dos filtros
  - pesos das camadas densas
- As funções de erro mais usadas são o EQM, para tarefas de regressão, e a entropia-cruzada, para tarefas de classificação.

Como podemos mitigar o  
sobreajuste em modelos  
profundos?

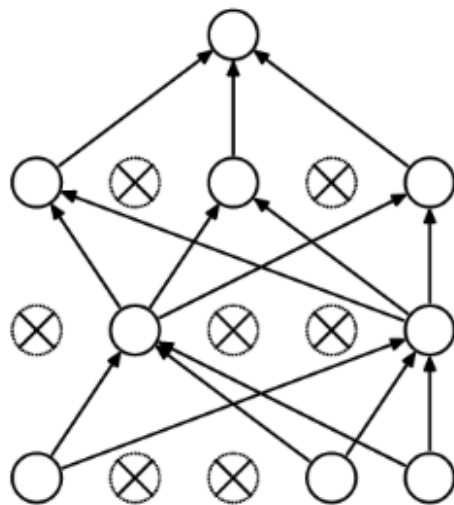
# Como evitar o sobreajuste

- Além de aumentar o *dataset*, reduzir a complexidade do modelo e aplicar técnicas de regularização como *early-stopping*, podemos usar três outras técnicas para evitar o *overfitting*:
  - *Dropout*
  - *Data augmentation*
  - *Transfer learning*
- Essas técnicas se aplicam a qualquer tipo de rede neural.

# Dropout



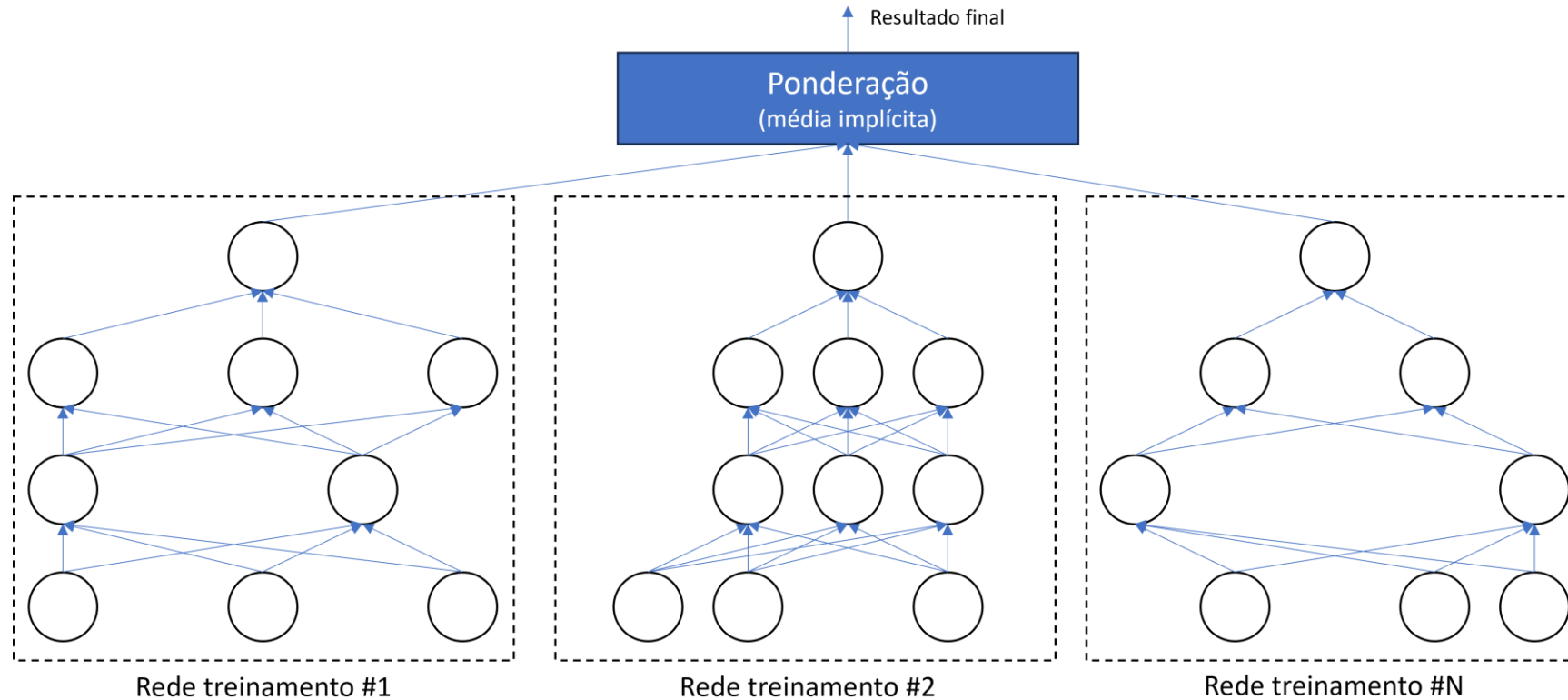
(a) Standard Neural Net



(b) After applying dropout.

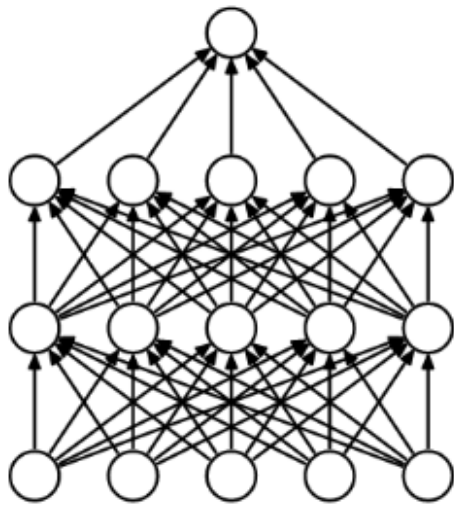
- O **dropout** é uma técnica de regularização.
- Envolve “**desligar**” aleatoriamente **neurônios de uma camada durante o treinamento** da rede neural.
- Isso significa que esses neurônios não contribuirão para o cálculo das saídas da rede durante uma iteração de treinamento.
- Desta forma, o **modelo** fica **menos complexo, diminuindo** as chances de **sobreajuste**.

# Dropout

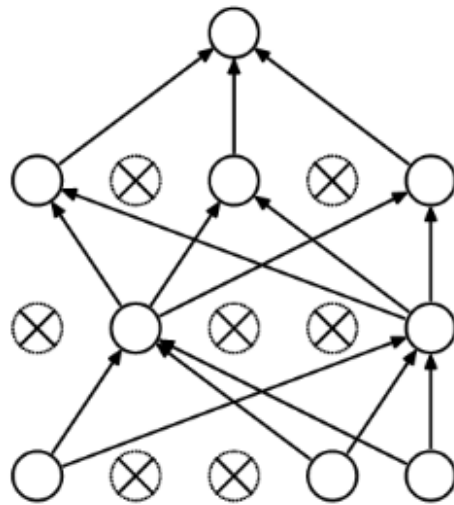


- Quando **desligamos** diferentes conjuntos de **neurônios**, é como se **estivéssemos treinando várias redes** neurais diferentes.

# Dropout



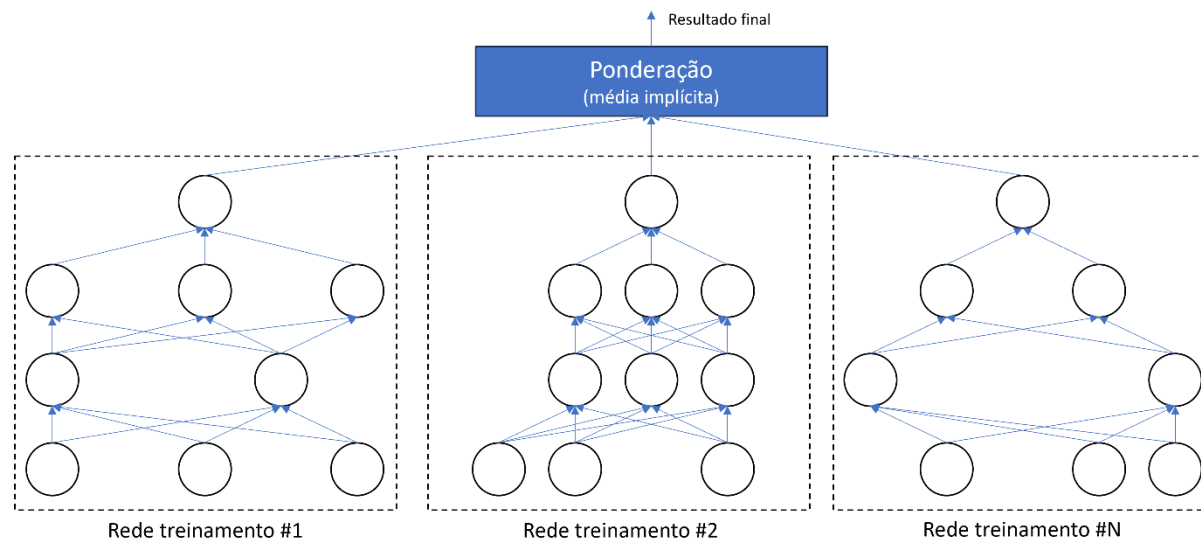
(a) Standard Neural Net



(b) After applying dropout.

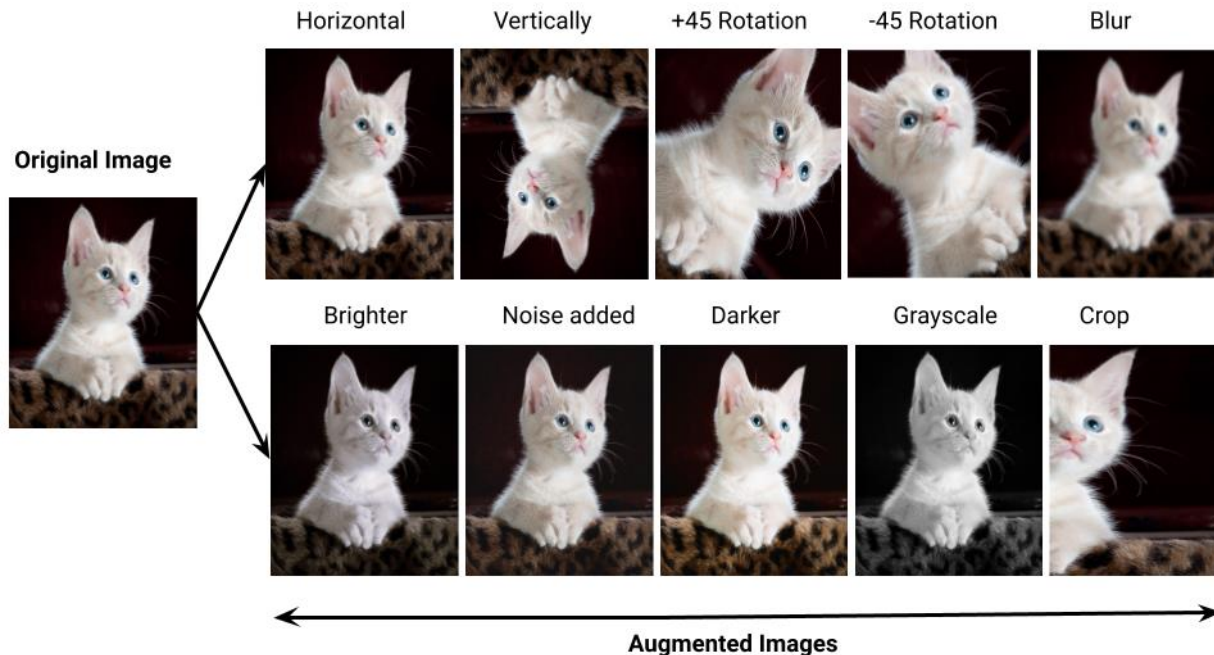
- Com *dropout*:
  - Cada neurônio precisa aprender de forma mais **robusta**
  - A rede não pode depender de combinações fixas
- Um neurônio precisa funcionar bem mesmo se outros neurônios desaparecerem.
- Isso força a rede a aprender **representações mais generalizáveis**.

# Dropout



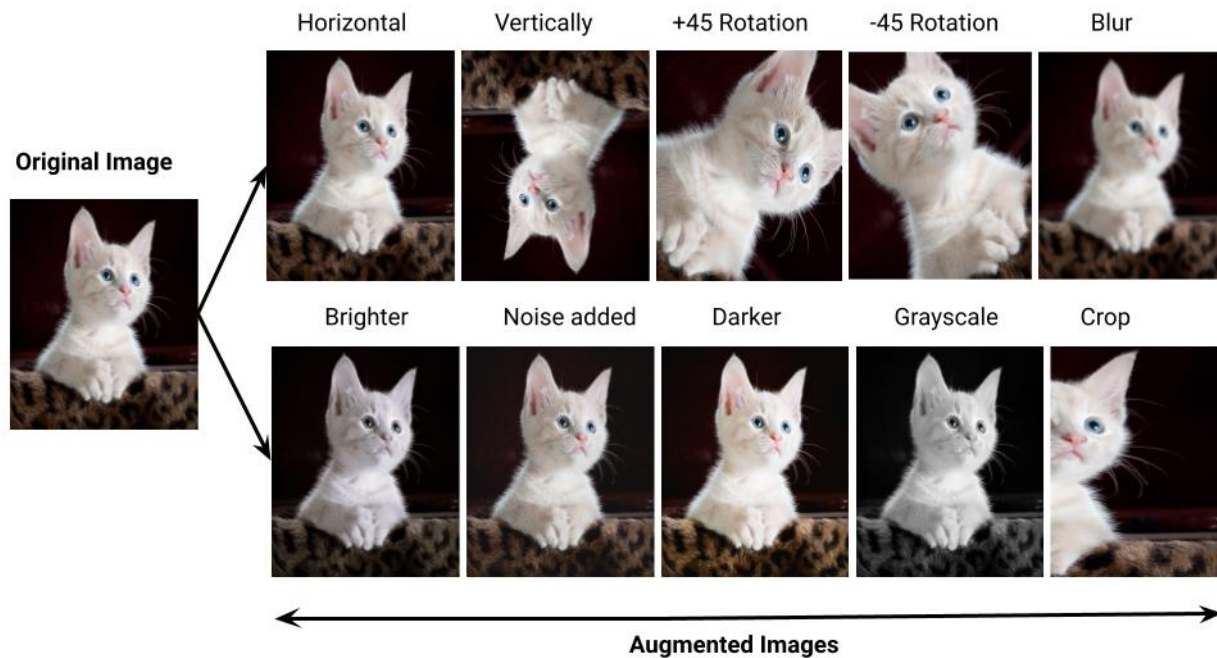
- Durante a **inferência**, o *dropout* não é aplicado (todos neurônios são usados).
- **Mesmo usando apenas uma única rede na inferência**, o comportamento dela é equivalente a uma **média de muitas redes diferentes** treinadas com *dropout*.
- As **diferentes redes se combinam** para reduzir o efeito do **sobreajuste**, generalizando melhor.

# Data augmentation



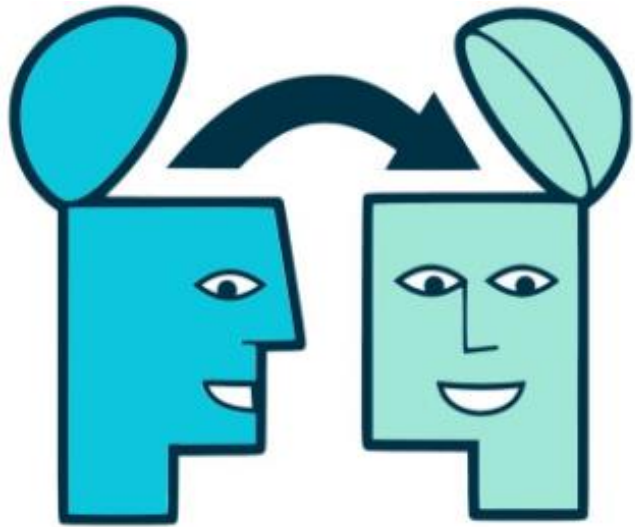
- Podemos aumentar artificialmente os dados.
- O objetivo é **aumentar** a **variedade** e a **diversidade** dos dados de treinamento **criando novos exemplos a partir dos dados existentes**.
- Essa técnica ajuda a melhorar o desempenho do modelo, tornando-o mais **robusto** e capaz de **generalizar melhor**.

# Data augmentation



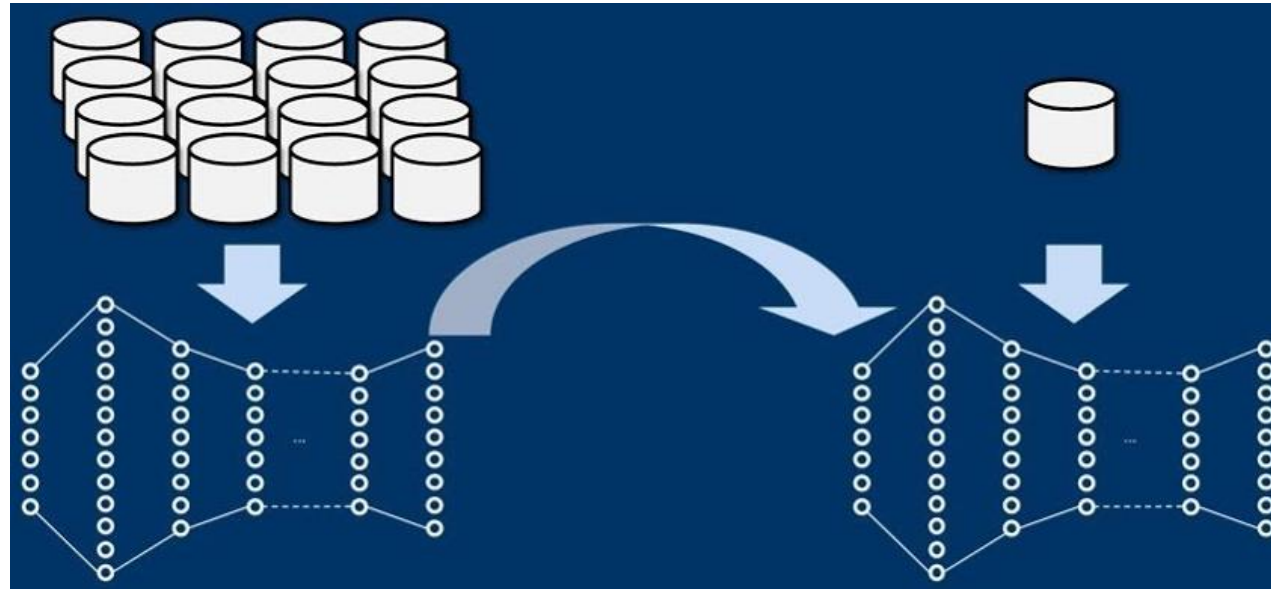
- Quando o problema envolve **imagens**, podemos aplicar **transformações**, como **rotação**, **espelhamento** e **corte**, modificação do **brilho**, adição de **ruído**, **desfoque**, etc.
- O **data augmentation** torna **CNNs** **robustas à rotação das imagens**, pois a rede é treinada com a **mesma imagem com diferentes rotações**.
- Assim a rede aprende a **reconhecer padrões independentemente da orientação**.

# *Transfer learning*



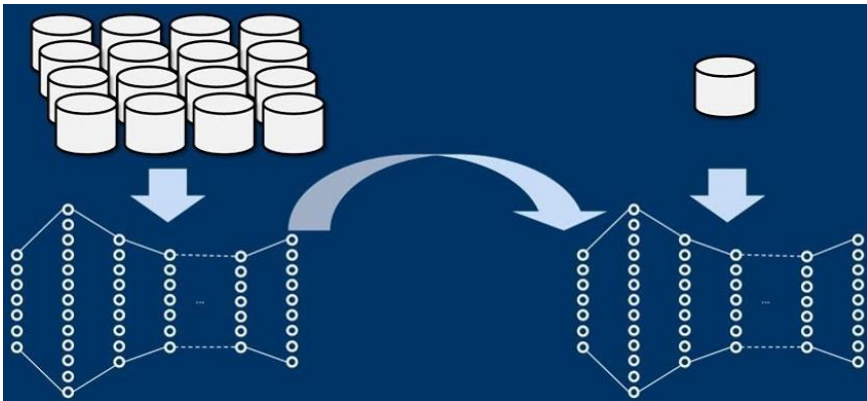
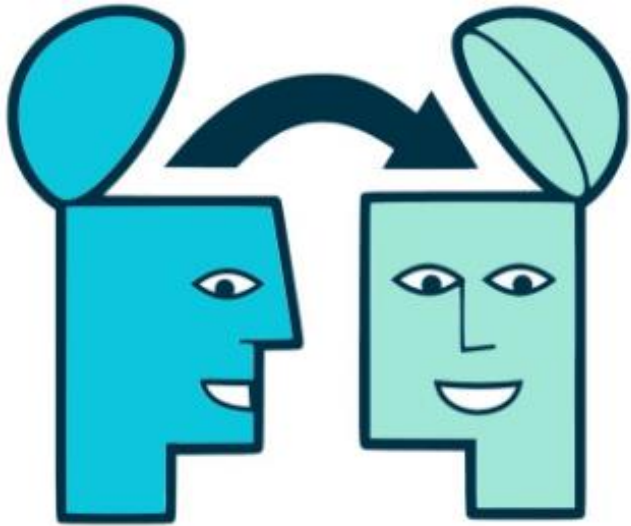
- *Transfer learning* ou **transferência de aprendizado** é uma abordagem que envolve o uso de um **modelo pré-treinado em uma tarefa** (ou domínio) relacionada **como ponto de partida para uma nova tarefa**.

# Transfer learning



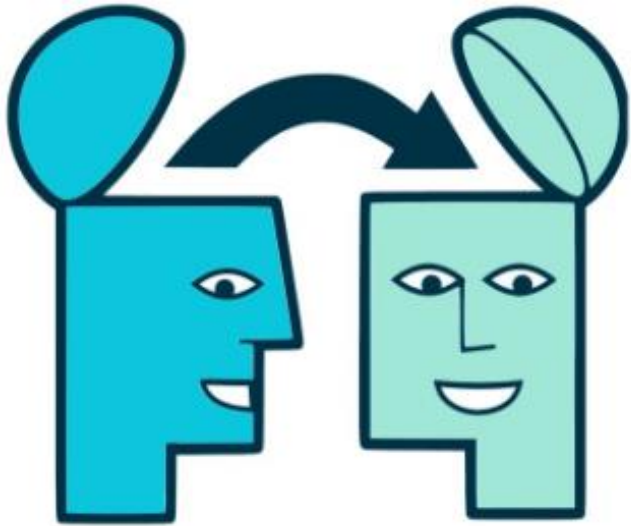
- Usamos o **conhecimento adquirido** por um **modelo treinando em uma grande base de dados** como **ponto de partida** para treinar um modelo com uma **base de treinamento menor, mas que seja relacionada à primeira**.

# Transfer learning

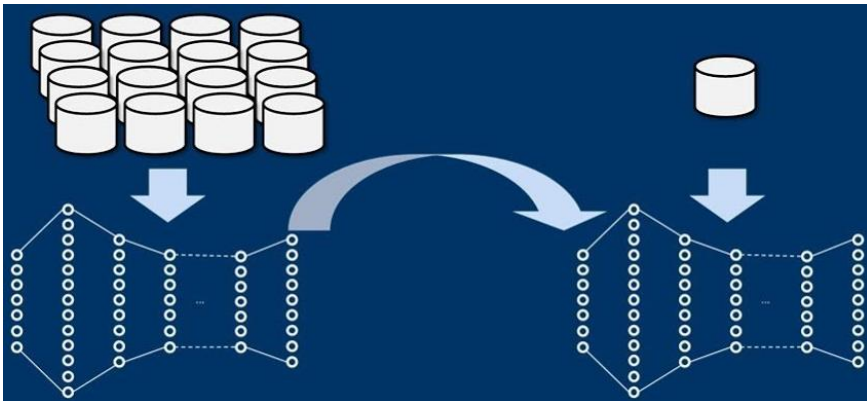


- Suponham que temos uma **rede pré-treinada** para classificar imagens em 100 categorias diferentes, incluindo **animais, plantas, veículos**, etc. e queremos treinar uma **nova rede para classificar tipos específicos de veículos**.
- Por essas tarefas serem semelhantes, **podemos reutilizar partes da primeira rede**.

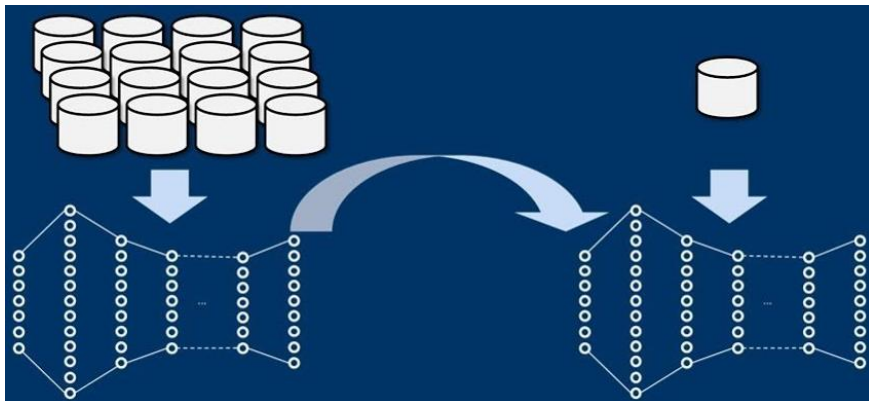
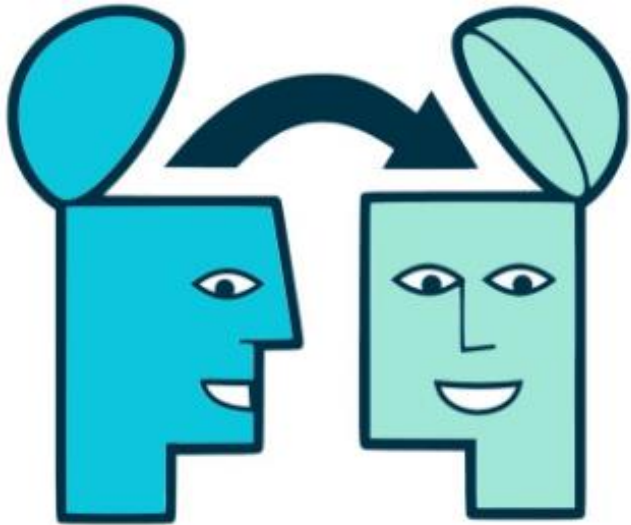
# *Transfer learning*



- Modelos pré-treinados em grandes conjuntos de dados **extraem características relevantes dos dados**.
- Essas **características são transferidas para o novo modelo**, que pode se beneficiar de uma **inicialização mais informada**.

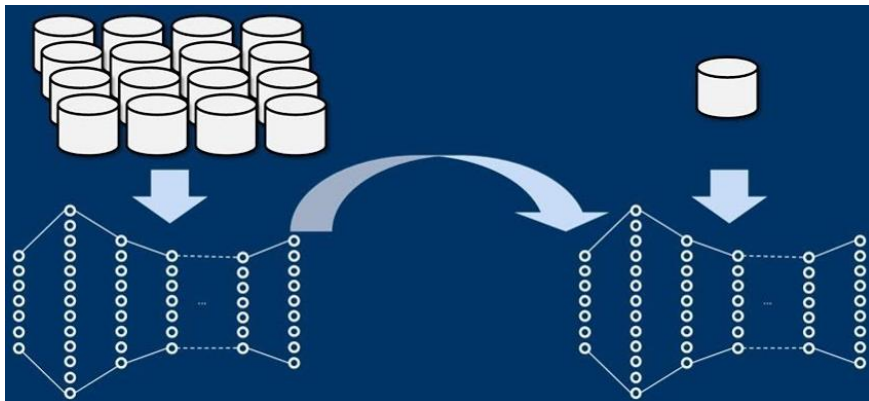
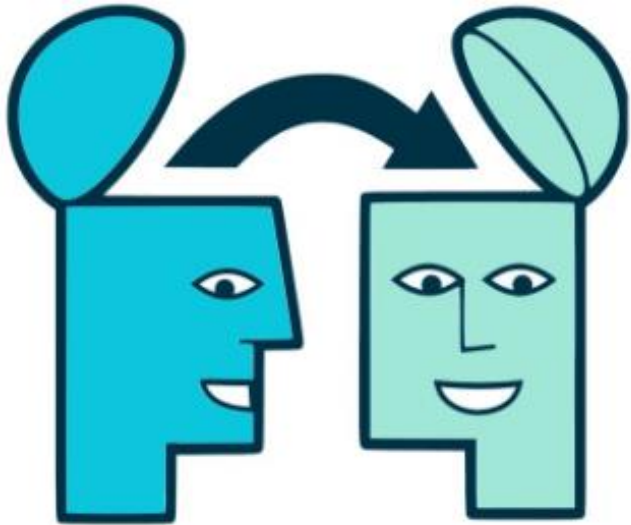


# Transfer learning



- O *transfer learning* **pode evitar** que o **novo modelo sobreajuste**, pois ele **já começa com características relevantes**.
- Como o **modelo pré-treinado já adquiriu um bom entendimento de características genéricas** em uma tarefa relacionada, o **treinamento do novo modelo pode exigir menos dados** para alcançar um bom desempenho.

# Transfer learning



- O *transfer learning* é particularmente **útil quando os dados de treinamento são escassos**, o que é uma situação **propensa ao sobreajuste**.
- Além de requerer menos dados, o *transfer learning* **acelera consideravelmente o treinamento**.

# Exemplo

- [Exemplo: cnn.ipynb](#)



# Lista de exercícios

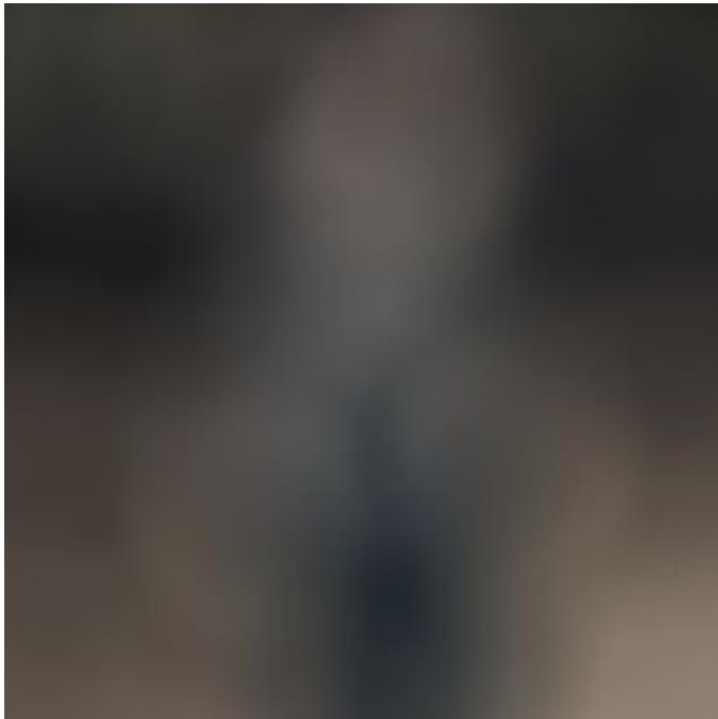
[Lista #10 - CNN](#)

Perguntas?

Obrigado!

# Anexo I: Exemplo de classificação de imagens

# Exemplo de classificação de imagens



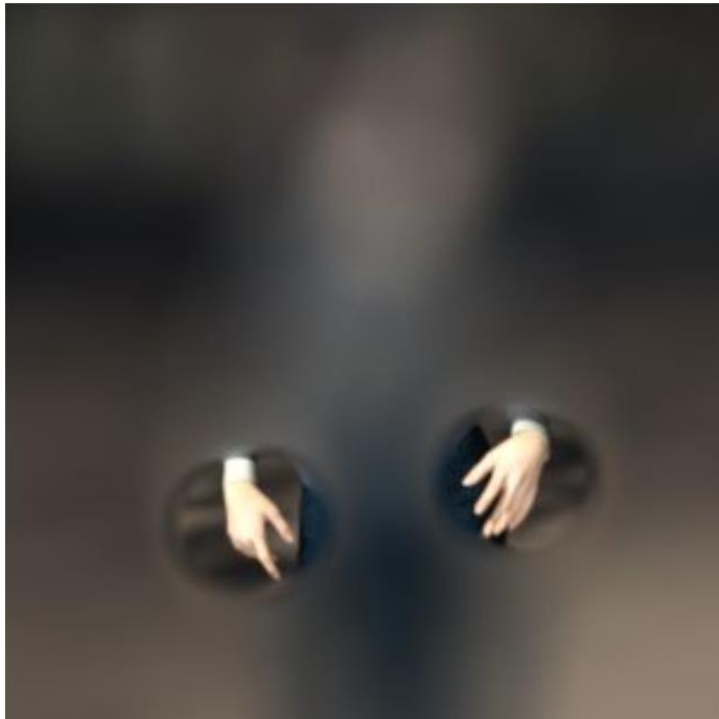
- Para entender melhor o que uma camada de convolução faz, vamos considerar um exemplo simples.
- Vamos supor que **não conhecemos o conteúdo da imagem** ao lado.
- Isso foi simulado deixando-a embaçada.

# Exemplo de classificação de imagens



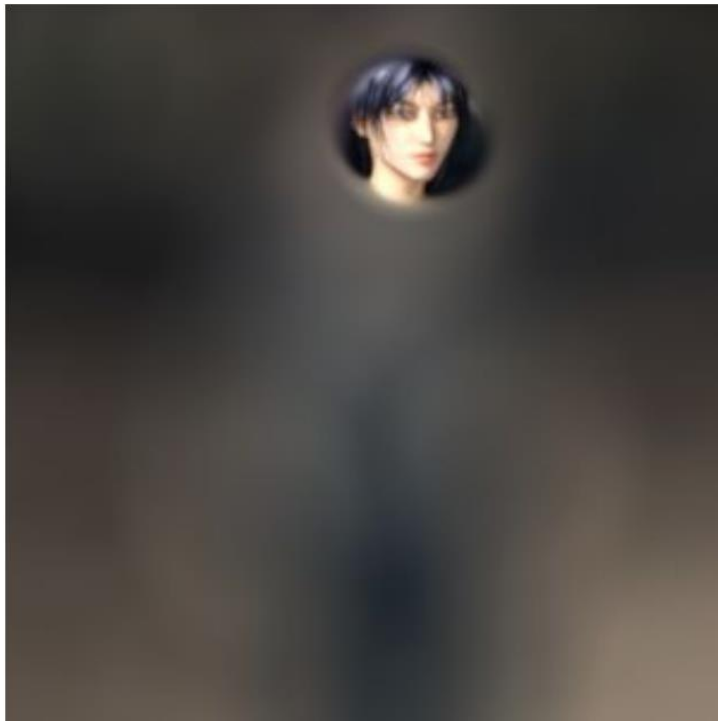
- Então, digamos que haja um **filtro** ou um **conjunto de filtros** que ao serem passados sobre a imagem **extraem as características** mostradas ao lado.
- Como podemos ver, **são duas formas verticais**, que se parecem com pernas humanas.

# Exemplo de classificação de imagens



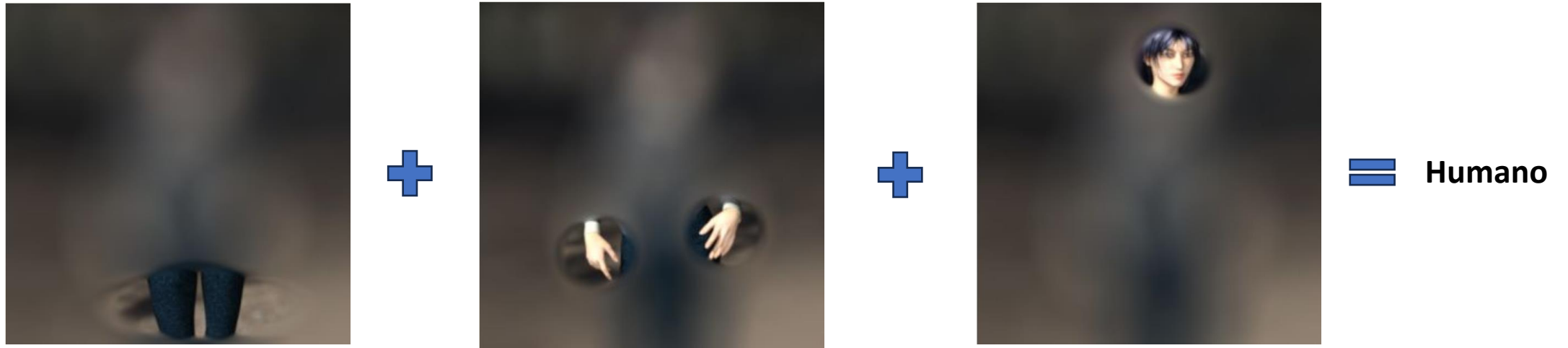
- Na sequência, **outros filtros extraem as características** mostradas ao lado.
- São pequenas formas cilíndricas que se projetam a partir de um cilindro maior.
- Nosso **cérebro vê isso** e **instantaneamente classifica como mãos**.
- Porém uma rede neural em treinamento ainda não sabe disso.
- Ela apenas sabe que um filtro pode extrair essas características.

# Exemplo de classificação de imagens



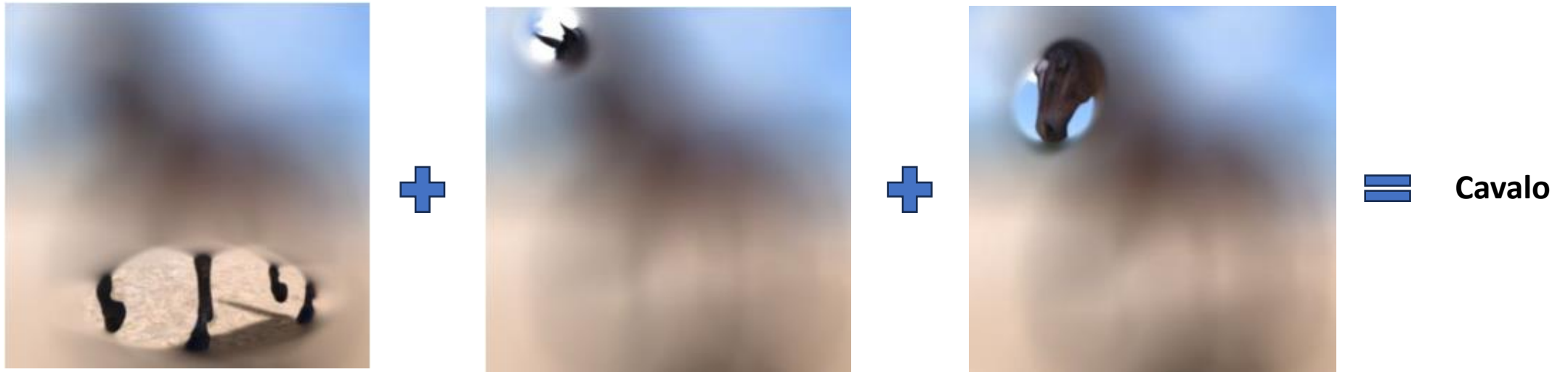
- Em seguida, **outros filtros extraem um círculo com outros dois círculos menores, uma saliência e algumas formas paralelas.**
- Nosso cérebro reconhece esse conjunto de pixels como um rosto com olhos, boca e nariz.
- Mas, novamente, o modelo não tem contexto para decidir o que esses pixels são.
- Ele só sabe que um filtro específico pode extrair essas características.

# Exemplo de classificação de imagens



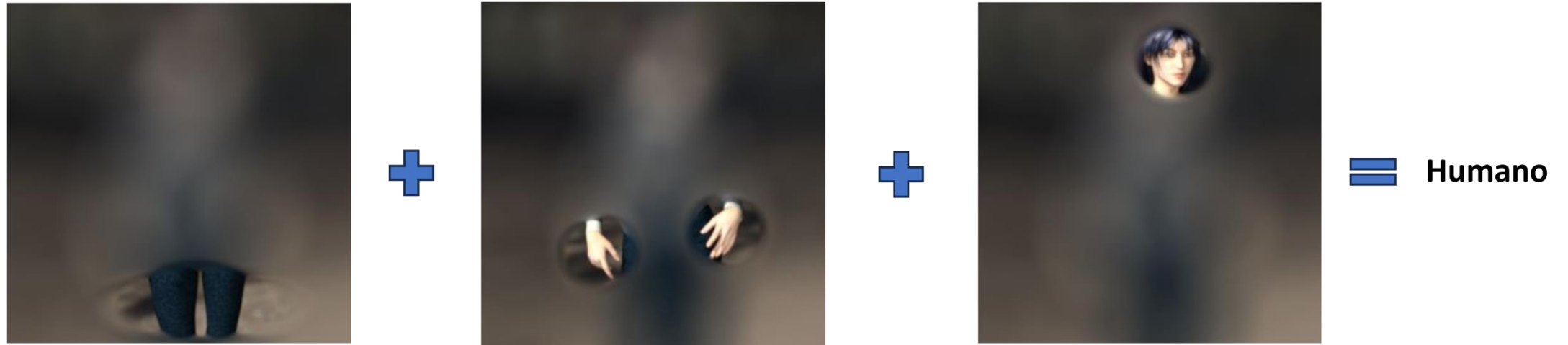
- Quando essas **características estão presentes em uma imagem rotulada como humana**, a rede neural pode ser **treinada para aprender filtros que extraem essas informações**.
- A rede **pode combinar as características extraídas pelos três filtros** (i.e., pernas, mãos e face) para detectar seres humanos em imagens inéditas.
  - A(s) camada(s) densa(s) faz(em) essa combinação.

# Exemplo de classificação de imagens



- A **mesma rede pode ser treinada para identificar** diferentes **características** presentes nas imagens de **cavalos**.
- Assim, ao **aprender conjuntos de filtros que podem detectar humanos ou cavalos**, temos um **modelo de visão computacional** que pode **lidar** com **imagens complexas** e **predizer o que há nelas**.

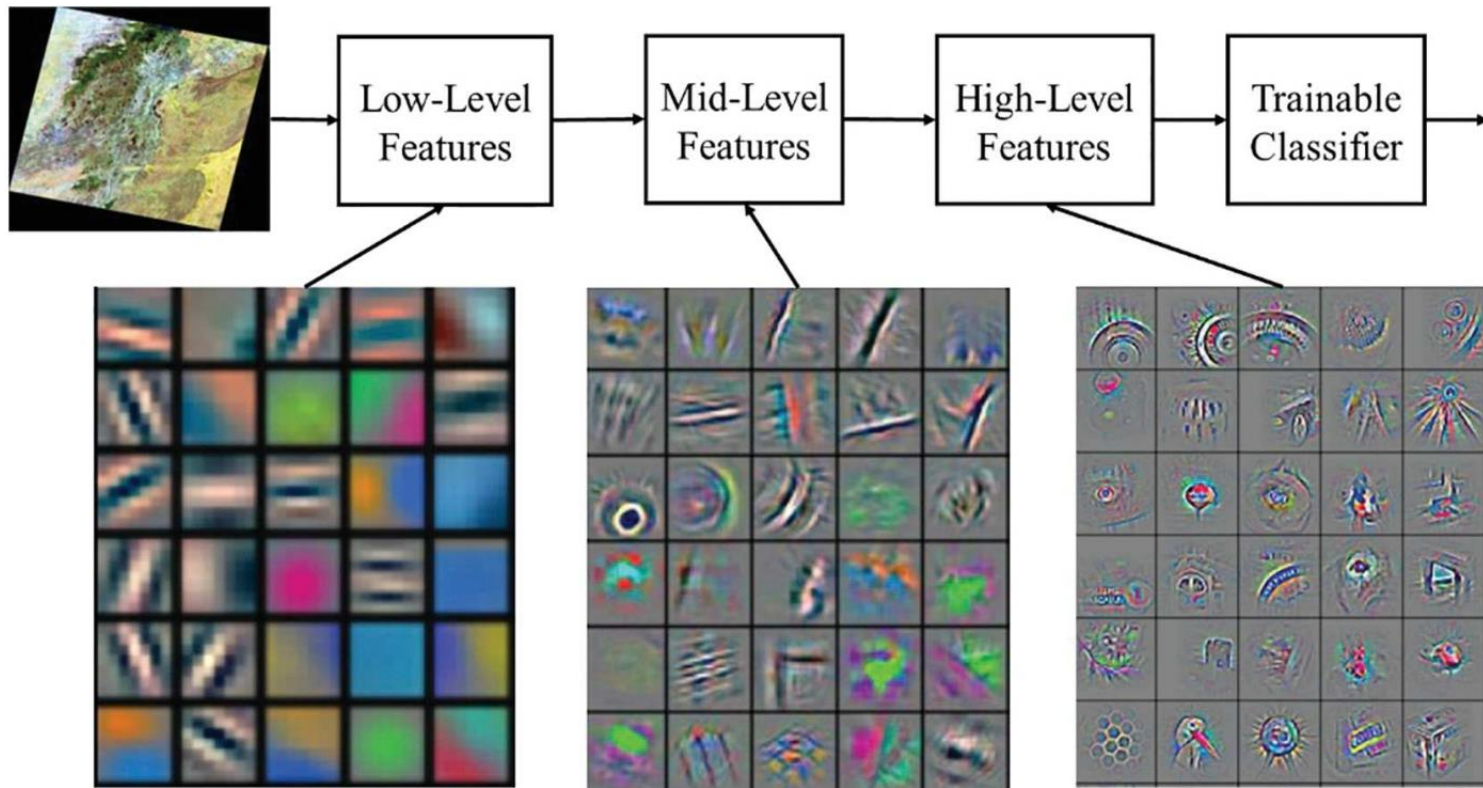
# Observação



- Notem que usamos pernas, mãos e rostos como exemplos de características extraídas por um filtro.
- Essas são formas reconhecíveis pelo nosso cérebro, então as utilizamos para ilustrar o conceito.
- Porém, as redes convolucionais, muito provavelmente **aprenderão características imperceptíveis** a nós humanos.

# Anexo II: Links interessantes

# Características imperceptíveis



- No entanto, **quando filtros são treinados** para detectar características em imagens, **eles podem identificar coisas que são imperceptíveis para os seres humanos.**
- Podem existir padrões de *pixels* que correspondam a uma imagem rotulada, mas que não tenham significado aparente para nós.

# Explorando CNNs

- CNN Explainer
  - <https://poloclub.github.io/cnn-explainer/>
- ConvNetJS MNIST demo
  - <https://cs.stanford.edu/people/karpathy/convnetjs/demo/mnist.html>
- ConvNetJS CIFAR-10 demo
  - <https://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>

Figuras

Input

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 0 | 1 |
| 2 | 3 | 4 | 5 |

$\otimes$

Kernel

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

$=$

Output

|    |  |
|----|--|
| 24 |  |
|    |  |



Input

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 0 | 1 |
| 2 | 3 | 4 | 5 |

$\otimes$

Kernel

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

$=$

Output

|    |    |
|----|----|
| 24 | 36 |
|    |    |



Input

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 2 | 3 |
| 4 | 5 | 6 | 7 |
| 8 | 9 | 0 | 1 |
| 2 | 3 | 4 | 5 |

$\otimes$

Kernel

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

$=$

Output

|    |    |
|----|----|
| 24 | 36 |
| 22 |    |

Input

|   |   |   |
|---|---|---|
|   |   |   |
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

$3 \times 3 \times 3$

Kernel

|   |   |
|---|---|
|   |   |
| 0 | 1 |
| 2 | 3 |

$2 \times 2 \times 3$

$\otimes$

=

Input

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

|   |   |   |
|---|---|---|
| 3 | 1 | 5 |
| 0 | 2 | 7 |
| 8 | 4 | 9 |

|   |   |   |
|---|---|---|
| 9 | 2 | 3 |
| 3 | 1 | 4 |
| 8 | 0 | 5 |

Kernel

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

$\otimes$

+

|   |   |
|---|---|
| 5 | 6 |
| 1 | 0 |

$\otimes$

+

|   |   |
|---|---|
| 1 | 3 |
| 4 | 2 |

$\otimes$

Output

|    |  |
|----|--|
| 69 |  |
|    |  |

$2 \times 2$

Input

Kernel

Input

Kernel

Output

|   |   |   |  |
|---|---|---|--|
|   |   |   |  |
|   |   |   |  |
| 0 | 1 | 2 |  |
| 3 | 4 | 5 |  |
| 6 | 7 | 8 |  |

$\otimes$

|   |   |  |
|---|---|--|
|   |   |  |
|   |   |  |
| 0 | 1 |  |
| 2 | 3 |  |

=

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

$\otimes$

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

+

$\otimes$

|   |   |
|---|---|
| 5 | 6 |
| 1 | 0 |

+

$\otimes$

|   |   |
|---|---|
| 1 | 3 |
| 4 | 2 |

=

|    |    |
|----|----|
| 69 | 85 |
|    |    |

$3 \times 3 \times 3$

$2 \times 2 \times 3$

$2 \times 2$

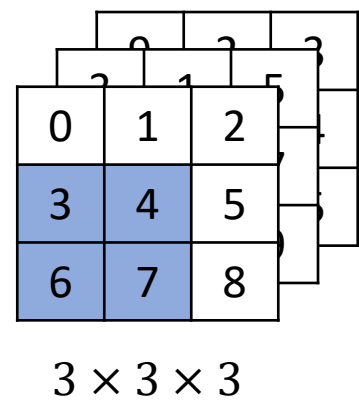
Input

Kernel

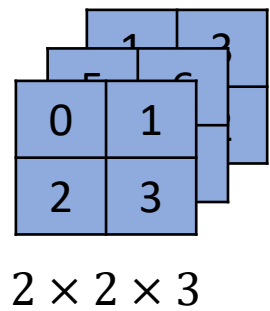
Input

Kernel

Output



$\otimes$



=

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

$\otimes$

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

+

$\otimes$

|   |   |
|---|---|
| 5 | 6 |
| 1 | 0 |

+

$\otimes$

|   |   |
|---|---|
| 1 | 3 |
| 4 | 2 |

=

|    |    |
|----|----|
| 69 | 85 |
| 95 |    |

$2 \times 2$

Input

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

$3 \times 3 \times 3$

$\otimes$

Kernel

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

$2 \times 2 \times 3$

=

Input

|   |   |   |
|---|---|---|
| 0 | 1 | 2 |
| 3 | 4 | 5 |
| 6 | 7 | 8 |

$\otimes$

Kernel

|   |   |
|---|---|
| 0 | 1 |
| 2 | 3 |

+

$\otimes$

|   |   |
|---|---|
| 5 | 6 |
| 1 | 0 |

+

$\otimes$

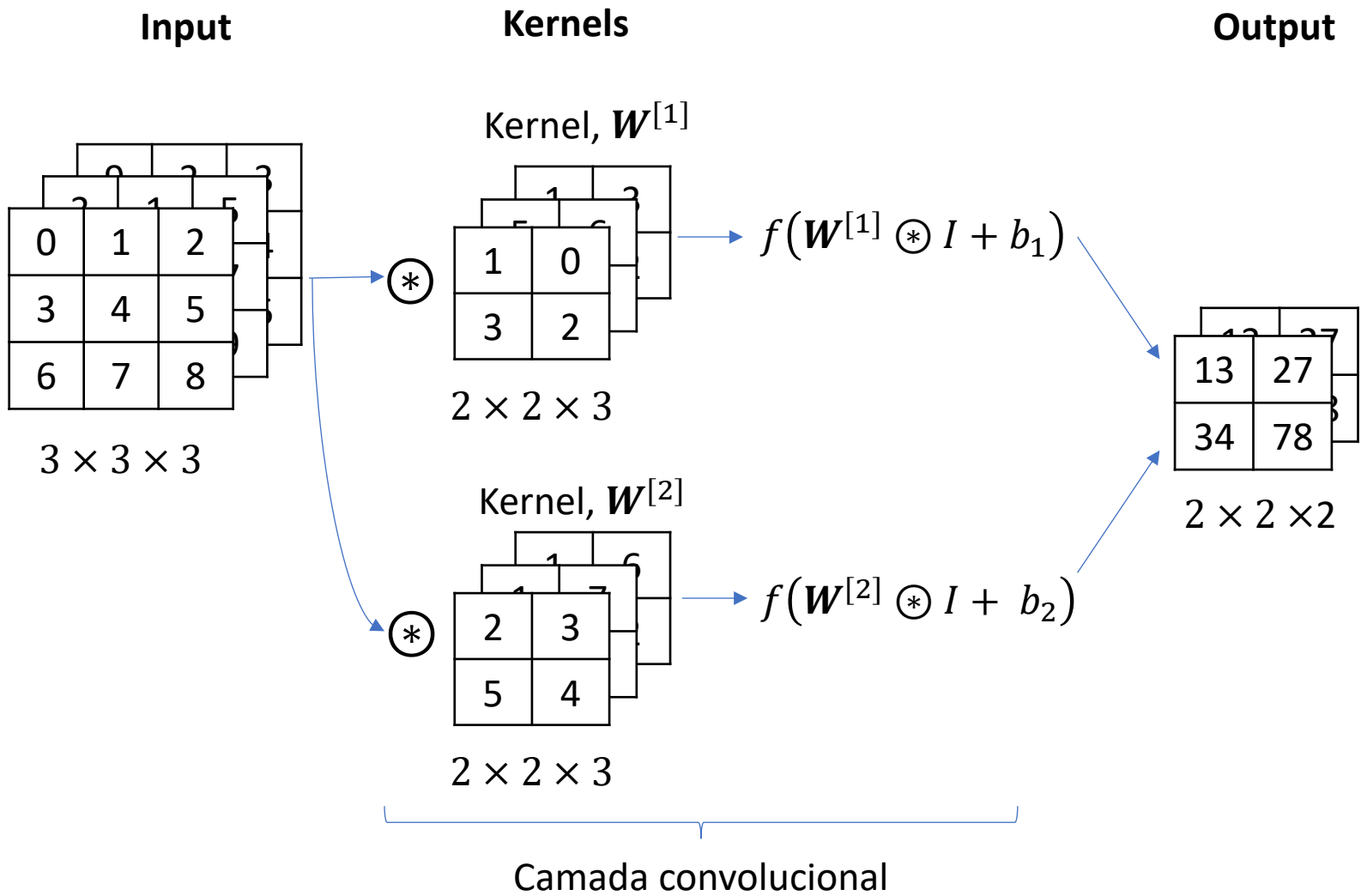
|   |   |
|---|---|
| 1 | 3 |
| 4 | 2 |

=

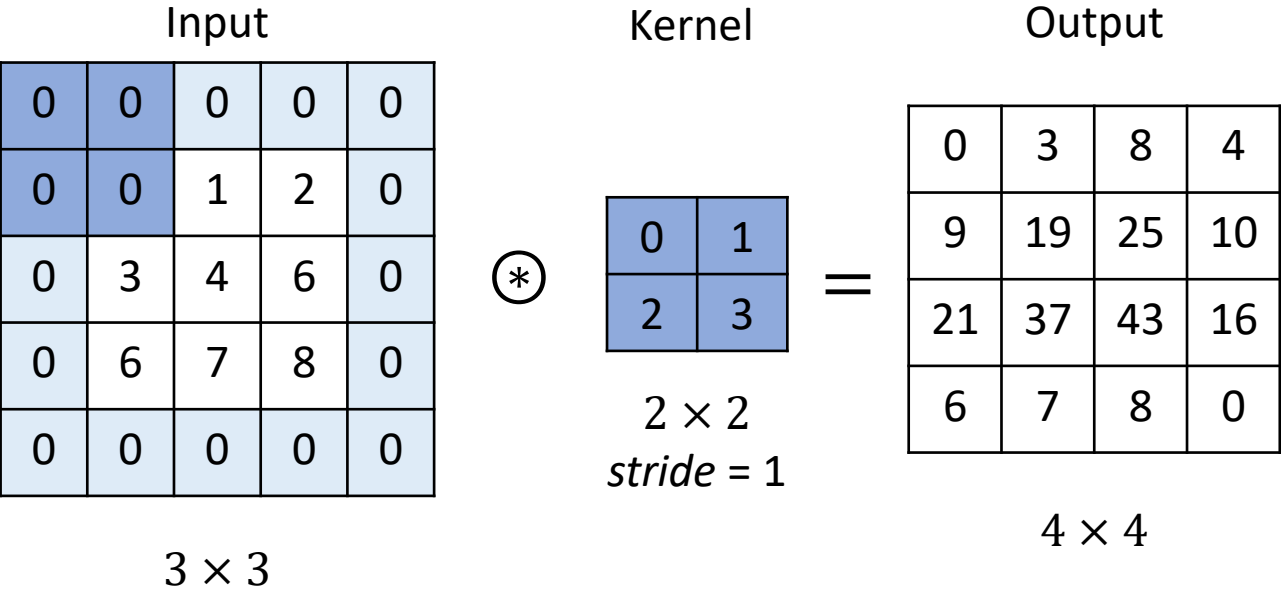
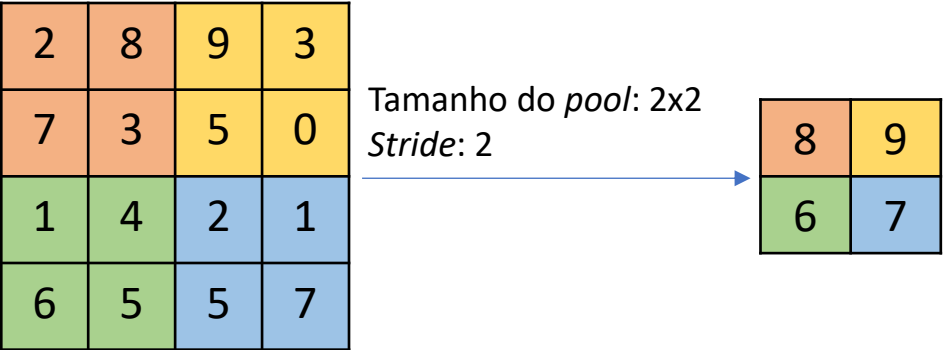
Output

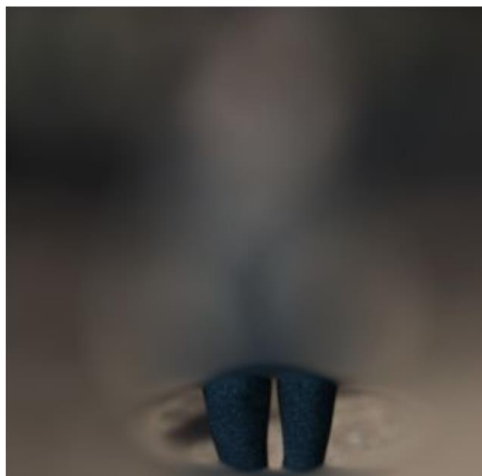
|    |     |
|----|-----|
| 69 | 85  |
| 95 | 122 |

$2 \times 2$

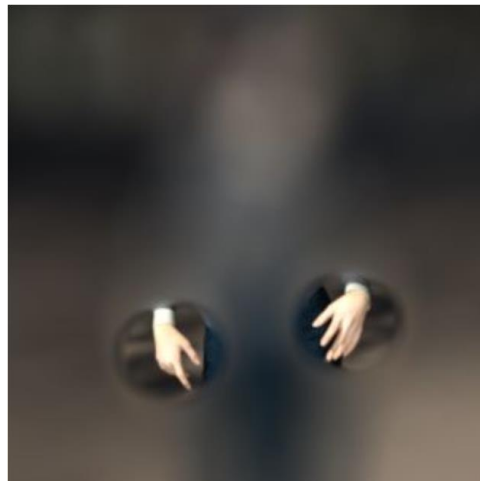


Max Pooling

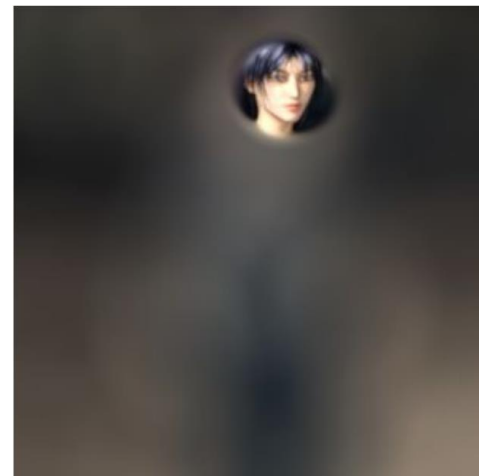




+



+



=

**Humano**

